

ACTION OMNI MULTI-MODAL MODEL THAT INTERACTS WITH
PHYSICAL WORLD

1. SITTIPON KUMDA
2. RACHATA PRATHEEP NA THALANG
3. SIPPAPHAT PITIWAN
4. TANAKORN YOUNGMEESUK
5. THANCHANOK NUENGCHAMNONG

A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENT FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN ROBOTICS AND AI ENGINEERING
SCHOOL OF ENGINEERING
KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG
2025
KMITL-20XX-EN-X-XXX-XXX

Thesis	ACTION OMNI MULTI-MODAL MODEL THAT INTERACTS WITH PHYSICAL WORLD
Students	Sittipon Kumda Rachata Pratheep Na Thalang Sippaphat Pitiwan Tanakorn Youngmeesuk Thanchanok Nuengchamnong
Student IDs	65011539, 65011499, 65011525, 65011563, 65011586
Degree	Bachelor of Engineering
Program	Robotics and AI Engineering
Year	2026
Thesis Advisor	Asst. Prof Dr. Poom Konghuayrob

ABSTRACT IN ENGLISH

This thesis presents a Vision-Language-Action (VLA) framework for robot arm manipulation in which visual perception, language understanding, and action generation are integrated into a single control pipeline. The work addresses key limitations of current VLA practice in low-resource settings: high data collection cost, limited access to high-memory hardware, and fragmented integration between multimodal AI and robot-controller execution. For practical deployment, VLA provides a clear benefit: robots can follow natural-language commands directly while reducing task-specific manual programming, and the same model can adapt more robustly to varied task conditions through unified vision-language-action reasoning.

The proposed architecture combines three coordinated stages: a QwenVL3.5 2B perception core for instruction-grounded scene understanding, a DiT Action Expert for executable robot action generation, and a safety layer (constraint check plus protocol adapter) for validated controller dispatch. The system receives an RGB scene observation and a free-form natural-language command, then produces structured manipulation actions suitable for safe robot-controller execution through a TCP/IP communication adapter. The design specifically targets a commercial setup with limited dataset volume and an 8 GB VRAM class training budget.

Experiments on representative robot manipulation tasks show that explicit QwenVL3.5 2B + DiT integration with safety-layer validation improves action validity and task success compared with partial baselines. In a direct comparison, extbfBaseline (Perception only) achieved extbf68.9

and extbf81.5

achieved extbf84.6

For data usage, this thesis used a simulation-only training set with extbf100 episodes (about extbf1

(

times

) recording duration in this work), whereas typical SOTA-style collection setups often use approximately extbf2

(

times

) longer recording duration. Evaluation across simulation and real-robot domains, including zero-shot settings, shows that the system follows natural-language instructions reliably, maintains practical real-time latency, and retains competitive performance under constrained training resources. The results support the feasibility of practical low-resource VLA-based robot control under realistic computation, data, and integration constraints.

Keywords: Vision-Language-Action, Robot Manipulation, Natural-Language Robot Control, Multimodal Integration, Generalization, Embodied AI

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Prof. Dr. Poom Konghuayrob, Our thesis advisor, for his patient guidance, constructive criticism, and consistent encouragement throughout every stage of this research. His deep knowledge of robotics and AI systems provided indispensable direction whenever this work encountered technical obstacles.

We are grateful to the faculty of the Robotics and AI Engineering programme at King Mongkut's Institute of Technology Ladkrabang for providing the academic foundation, the computational resources, and the laboratory environment that made this project possible.

We also wish to thank our fellow students and research colleagues for their valuable discussions, their willingness to review drafts, and their practical help during hardware assembly and testing.

Finally, we owe a profound debt of gratitude to my family for their unwavering support, patience, and encouragement throughout my undergraduate studies. This work is dedicated to them.

(65011539) Sittipon Kumda

(65011499) Rachata Pratheep Na Thalang

(65011525) Sippaphat Pitiwan

(65011563) Tanakorn Youngmeesuk

(65011586) Thanchanok Nuengchamnong

TABLE OF CONTENTS

	Page
Abstract	ii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS	vii
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS AND SYMBOLS	xi
 CHAPTER 1 INTRODUCTION	1
1.1 Statement and Significance of the Problems	1
1.2 Goal and Objectives of the Study	2
1.3 Scope of the Study	2
1.4 Process of the Study	3
1.5 Limitations of the Study	3
1.6 Definitions	4
 CHAPTER 2 RELATED THEORIES AND LITERATURE REVIEW	5
2.1 ROBOT KINEMATICS AND CLASSICAL PIPELINES	5
2.1.1 Rigid Body Kinematics and FK/IK	5
2.1.2 Traditional Path Planning vs. VLA Cognitive Control	6
2.1.3 Practical Hardware Deployment: Dobot CR5 via Python	6
2.2 VISION-LANGUAGE MODELS (VLMs) IN EMBODIED AI	7
2.2.1 Native vs. Fusion Architectures	7
2.2.2 Contrastive Encoders and the Semantic-Spatial Gap	7
2.3 THE EVOLUTION OF VLA: FROM TOKENS TO EXPERTS	8
2.3.1 The Genesis: Next-Token Prediction	8
2.3.2 The Modern Era: Decoupled Action Policy Experts	8
2.4 DIFFUSION MODELS IN ROBOTICS CONTROL	8

2.4.1	DDPM and Action Chunking	8
2.4.2	U-Net vs. DiT Backbones	9
2.4.3	Flow Matching for Fast Inference	10
2.5	STATE-OF-THE-ART IN VLA INTEGRATION	10
2.5.1	OpenVLA	10
2.5.2	SmolVLA.....	11
2.5.3	Pi-Zero (π_0) and Pi0.5.....	11
2.6	GENERALIZATION AND CHAPTER SUMMARY	12
CHAPTER 3 IMPLEMENTATION		15
3.1	Phase 1: Robot to AI Communication.....	15
3.1.1	TCP/IP Interface and Byte Decoding.....	15
3.1.2	Tensorization and Storage	15
3.2	Phase 2: Dataset Preparation	15
3.2.1	Simulation Data Pipeline	15
3.2.2	Real-World Data Pipeline.....	15
3.2.3	Task Variance and Dataset Statistics.....	16
3.3	Phase 3: Model Architecture.....	16
3.3.1	Vision-Language Backbone	17
3.3.2	Overview-Aware Strategy.....	18
3.3.3	Structured Latent Conditioning via Masked Cross-Attention	18
3.3.4	Diffusion Transformer (DiT)	20
3.4	Phase 4: Training Objectives and Flow Matching.....	20
3.4.1	The Multi-Component Loss Function.....	21
3.5	Phase 5: Model Evaluation and Improvement	21
CHAPTER 4 EXPERIMENTAL RESULTS AND DISCUSSION.....		22
4.1	EXPERIMENTAL SETUP.....	22
4.1.1	Hardware and Runtime Environment	22
4.1.2	Task Protocol.....	22
4.1.3	Evaluation Domains	22
4.1.4	Metrics	22
4.1.5	Dataset Collection Context and SOTA Comparison.....	23
4.1.6	New-Data Diagnostics with Added Figures	23
4.2	END-TO-END VLA PIPELINE PERFORMANCE.....	28
4.3	INTEGRATION OF PERCEPTION, DiT, AND SAFETY LAYER.....	29
4.4	NATURAL-LANGUAGE INSTRUCTION FOLLOWING.....	31
4.5	GENERALIZATION AND ZERO-SHOT PERFORMANCE.....	32
4.6	TASK-WISE MANIPULATION PERFORMANCE.....	32

4.7	DESIGN CONSTRAINT VALIDATION	38
4.7.1	Computation and Memory	38
4.7.2	Latency	38
4.7.3	Integration Efficiency	39
4.8	VLM INTERPRETABILITY RESULTS.....	39
4.8.1	Dataset and Corpus.....	39
4.8.2	Feature Discovery	39
4.8.3	Activation Steering	39
4.9	DISCUSSION.....	40
CHAPTER 5	CONCLUSION AND FUTURE WORK	41
5.1	CONCLUSION.....	41
5.2	LIMITATIONS	41
5.3	FUTURE WORK	42
	BIBLIOGRAPHY	43
CHAPTER A	Supplementary Materials	45
A.1	System Configuration Summary	45
A.2	Additional Experimental Settings.....	45
A.3	Extended Results.....	45
A.4	Implementation Notes.....	46
A.5	External Tooling and Repository Links	46
A.6	Usage Instructions for Reproduction	46

LIST OF TABLES

Table	Page
3.1 Spatial Intelligence Benchmark Comparison across Qwen-VL Model Variants.	18
3.2 Breakdown of the Multi-Component Loss Function	21
4.1 Dataset collection setting in this thesis compare to common SOTA-style practice.	23
4.2 Summary of additional diagnostics for new data and reference data.	28
4.3 System-level output quality for end-to-end VLA inference.	28
4.4 Ablation results for module integration.	29
4.5 Natural-language instruction-following performance.	31
4.6 Seen vs unseen task performance.	32
4.7 Performance by manipulation task category.	33
4.8 Qualitative effect of activation steering at varying injection strength α	39
A.1 System configuration used in this study	45
A.2 Extended performance results	46

LIST OF FIGURES

Figure	Page
2.1 A diagram of Dobot Cr5	5
2.2 Comparison between U-Net and DiT	9
2.3 OpenVLA architecture.....	11
2.4 SmoVLA architecture	12
2.5 Pi0.5 architecture	13
3.1 Overall Vision-Language-Action pipeline. Multimodal inputs flow through the QwenVL 3.5 2B perception backbone and DiT Action Expert, generating safe, validated trajectories for the Dobot CR5	17
3.2 Detailed flow of the structured latent conditioning. A fixed set of trainable Latent tokens cross-attends to the QwenVL output. The refined tokens (Latent*) are combined with an orthogonally projected Robot State to form the final Context for the Diffusion Transformer.	19
3.3 Boolean cross-attention mask mapping. Shaded cells indicate permitted attention connections between the latent token chunks and the specific modality source tokens. Unshaded cells are masked.	20
4.1 Reference dataset metadata summary (episodes, recording duration, frame rate, and episode-length distribution) used for comparison context.....	24
4.2 Action autocorrelation of the new dataset. Suggested chunk length is 10 steps (≈ 0.33 s), indicating short-horizon temporal dependence.	25
4.3 State-action temporal alignment of the new dataset. Mean control delay is 1 step (≈ 0.033 s), indicating near-synchronous action-state coupling.	25
4.4 Demonstrator speed variance in the new dataset. Coefficient of variation is 0.289, indicating moderate speed dispersion across episodes.	25
4.5 Cross-episode action variance in the new dataset. Variance concentration in specific temporal regions indicates phase-structured manipulation behavior.	26
4.6 Reference-set temporal diagnostics (autocorrelation, state-action delay, and speed variance) for side-by-side interpretation against the new data.....	27
4.7 Reference-set cross-episode action variance map across bilateral joints.	27
4.8 Representative episode rollout for a pick-and-place instruction, showing joint trajectories and synchronized camera observations over time.	29
4.9 Action-token to context-token attention map in the final diffusion transformer block. Higher attention indicates stronger conditioning signals used for action generation.	30
4.10 Qwen image-token activation maps across transformer depth for task-focused scene understanding.	30

4.11	Context-embedding clustering before and after projection, showing improved separation by task type while preserving object-color structure.....	31
4.12	Autoregressive chunking consistency in overlap regions, demonstrating stable joint prediction across chunk boundaries.....	32
4.13	Trajectory segmentation under a motion threshold, illustrating moving and stationary phases used for temporal behavior analysis.....	33
4.14	Per-joint position and anchor-velocity distributions used to inspect action-space coverage and motion statistics.	34
4.15	Joint position correlation matrix computed from the LeRobot dataset split.	35
4.16	Joint position correlation matrix computed from the project evaluation set.....	36
4.17	Per-episode joint position range heatmap indicating variability by joint and episode.....	37
4.18	End-effector workspace coverage in 3D and projected density views, indicating explored reachable regions.	38

LIST OF ABBREVIATIONS AND SYMBOLS

C	Specific heat
P	Pressure
V	Volume

CHAPTER 1

INTRODUCTION

1.1 Statement and Significance of the Problems

The past decade has witnessed a profound transformation in artificial intelligence, driven by the emergence of Vision-Language Models (VLMs). These neural networks jointly process visual and textual inputs to produce semantically grounded outputs. By training on hundreds of millions of image-text pairs, these models have acquired broad and transferable understanding: they can recognise objects, infer spatial relationships, follow complex instructions, and reason about scenes they have never explicitly been trained on [13, 10, ?]. This generalized understanding has successfully unlocked a new tier of software automation, streamlining complex digital tasks such as zero-shot optical character recognition (OCR) and automated document summarization [1].

The natural progression of this capability is its extension from the digital domain into the physical world. If a VLM can interpret an image and respond to questions about it, it should be able to direct a robot to physically interact with the objects it perceives. This transition forms the foundation of Vision-Language-Action (VLA) models — systems that couple the semantic reasoning of a VLM with a physical action generation module, enabling a robot to receive a natural-language instruction, interpret the visual state of its environment, and produce motor commands to accomplish the specified goal [2, 4]. The potential for industrial automation is substantial: a robot that understands language and vision could be deployed flexibly across diverse tasks with minimal reprogramming, adapting to new objects and instructions without the task-specific engineering that classical robotic systems require [9].

However, while VLM research has flourished globally, the study and development of VLA systems remains remarkably narrow. This disparity is primarily driven by severe hardware and computational constraints. Although advanced VLA models such as RT-2 [2], OpenVLA [8], and GR00T N1 [11] have demonstrated impressive results, their development relies on resources that are inaccessible to most research groups: multi-GPU server clusters, purpose-built robot fleets costing tens of thousands of dollars, and datasets comprising hundreds of thousands of demonstration episodes [12, 7]. As a consequence, VLA research remains far narrower in participation and diversity than VLM research, and its findings are difficult to reproduce or extend in resource-constrained settings.

This concentration of capability matters because it slows educational adoption, weakens scientific reproducibility, and delays the transfer of embodied AI from research laboratories to practical applications. If practical design principles for VLA development under constrained conditions are not established, access to this technology will remain limited to well-funded institutions. To address this, the study proposes an end-to-end framework to develop a VLA model strictly on consumer-grade setups. By bridging the gap between high-end capability and low-resource accessibility, this framework aims to make VLA research reachable and reproducible worldwide.

The problem can be summarised in four concrete dimensions:

1. **High entry cost:** existing VLA pipelines require long-duration data collection, large task diversity, and multi-GPU infrastructure that is beyond the reach of most academic groups and small research teams.
2. **Insufficient low-resource methodology:** limited published guidance exists for training robust

VLA models under narrow data and compute budgets, and no open end-to-end framework currently targets consumer-grade hardware.

3. **AI-to-robot integration challenges:** practical deployment fails when model outputs cannot be reliably translated into safe, standardised, and executable robot commands at real-time control frequencies.

4. **Incomplete validation:** many studies emphasise simulation performance while real-robot behaviour, zero-shot transfer, and deployment constraints remain under-reported, limiting the practical relevance of their findings.

Addressing these issues is significant because it enables broader participation in embodied AI research, improves scientific reproducibility, and increases the practicality of language-conditioned robot systems for real operational environments beyond large research institutions.

1.2 Goal and Objectives of the Study

The primary goal of this study is to develop a practical, end-to-end Vision-Language-Action framework that operates effectively on consumer-grade hardware, thereby democratizing access to embodied AI research worldwide. This is achieved by designing a system that is explicitly bounded by consumer-grade hardware constraints from the outset, rather than treating resource efficiency as a secondary concern.

The specific objectives are:

1. To train a VLA model using the proposed end-to-end framework within an 8 GB VRAM budget, and to demonstrate practical manipulation capability by achieving a task success rate above 50%, improving clearly over the approximately 30% baseline.

2. To establish a robust VLA development framework that enables effective alignment between simulation data and real-world robot datasets, so that cross-domain learning, transfer reliability, and real-world deployment stability are improved under consumer-grade resource constraints.

1.3 Scope of the Study

The study focuses on language-guided tabletop manipulation using a Dobot CR5 six- degree-of-freedom robot arm. RGB visual observations from two fixed cameras and natural-language instructions are the primary system inputs. The task domain covers four manipulation categories: pick-and-place, stacking, drawer and container interaction, and multi-step sequencing.

The demonstration dataset is intentionally constrained to a low-resource regime, comprising approximately one hour of robot motion distributed across simulation and real-world sources. Simulation trajectories are generated in Isaac Lab with domain randomisation; real-world demonstrations are collected using the open-source Dobot Dataset Collection Tools [5] via human teleoperation.

Model development and training are designed to fit commercially accessible compute resources, with a target peak training budget of approximately 8 GB VRAM achieved through bfloat16 mixed precision, gradient checkpointing, and QLoRA [3]. Evaluation is conducted in both simulation and real-robot settings and includes success-rate assessment on seen tasks, zero-shot generalisation analysis on unseen object-instruction combinations, action validity measurement, and end-to-end latency profiling.

1.4 Process of the Study

The study is conducted through a structured end-to-end process:

1. **System design:** a modular VLA architecture is specified by integrating three core stages — a QwenVL3.5 2B perception backbone [?] for multimodal understanding, a trainable Diffusion Transformer Action Expert [14] for continuous trajectory generation, and a safety layer (constraint check plus protocol adapter) for verified robot dispatch. The architecture is designed to satisfy the 8 GB VRAM constraint from the outset.
2. **Data construction:** simulation trajectories from Isaac Lab and real-world teleoperation recordings are collected, synchronised, and converted into LeRobot-compatible corpora with per-joint normalisation and outlier filtering. The simulation corpus is used for training, while real-world data is reserved for evaluation and deployment validation.
3. **Model training:** the pipeline is trained in two stages. Stage 1 trains only the lightweight connector and action modules with frozen backbones, establishing stable action generation before backbone adaptation. Stage 2 applies LoRA fine-tuning [6] to the VLM backbone with reduced learning rate to adapt language-visual grounding to the manipulation domain without catastrophic forgetting.
4. **Robot integration:** model outputs are translated into executable robot commands through a safety-aware protocol adapter that performs de-normalisation, constraint validation, and TCP/IP dispatch to the Dobot CR5 controller. A three-tier failure-handling protocol (rescale, abort, episode reset) ensures safe operation under constraint violations.
5. **Evaluation and analysis:** performance is measured across simulation and real-robot settings using four metrics — task success rate, action validity, generalisation ratio, and end-to-end latency. Ablation experiments isolate the contribution of each architectural module. Failure cases are categorised by stage (perception, language grounding, action decoding) to guide targeted improvements.
6. **Iterative refinement:** error patterns identified from evaluation inform targeted improvements including data reweighting, LoRA rank adjustment, prompt engineering, and action-chunk horizon tuning, applied to improve reliability and deployment feasibility within the resource constraints.

1.5 Limitations of the Study

Several limitations are acknowledged in this work. First, task diversity is restricted to four tabletop manipulation categories using a small set of object types; conclusions regarding task generalisation should not be interpreted as evidence of broad universality across the full range of manipulation scenarios.

Second, the demonstration dataset is intentionally small — approximately one hour of motion data, with simulation-only training and real-robot deployment evaluation — which may constrain robustness when facing highly diverse object configurations or unconventional instruction phrasings not represented in the training distribution.

Third, the evaluation platform is restricted to a single robot type (Dobot CR5); cross-platform transfer to robots with different kinematics, actuation mechanisms, or control interfaces is not validated in the present study. Fourth, all experiments are conducted under controlled laboratory conditions with consistent lighting and known object categories. Large-scale industrial deployment, human-safety certification, and long-horizon

autonomous operation are outside the scope of this thesis.

Finally, the VLM backbone is adapted through LoRA rather than full parameter fine-tuning, which limits the degree of task-specific adaptation achievable within the compute budget and may result in residual misgrounding on instruction phrasings that differ substantially from the backbone’s pre-training distribution.

1.6 Definitions

The following definitions are used consistently throughout this thesis:

1. **Vision-Language Model (VLM):** a multimodal neural network that jointly processes visual observations and textual inputs within a shared representational framework, enabling instruction interpretation, visual reasoning, and natural-language response generation [10, 13].
2. **Vision-Language-Action (VLA) model:** an extension of the VLM paradigm that additionally produces executable physical control actions from multimodal inputs, enabling an embodied agent to follow language instructions by directly manipulating objects in its environment [2, 8].
3. **End-to-end pipeline:** an integrated learning and deployment system in which perception, instruction understanding, and action generation are trained and executed as a single continuous process, without relying on manually engineered intermediate representations between stages.
4. **Low-resource setting:** an operational condition characterised by constrained data volume and limited compute resources. In this study, the low-resource setting is defined as approximately one hour of robot motion data and a peak training and inference budget of approximately 8 GB VRAM on a single consumer GPU.
5. **Zero-shot generalisation:** the capability of a trained model to perform correctly on tasks or instruction variants that were not explicitly observed during training, relying on the generalisation capacity of its pre-trained backbone rather than task-specific memorisation.
6. **AI-to-robot interface:** the communication and safety layer that converts the continuous action vectors produced by the model into hardware-compatible robot commands, applying constraint validation and protocol formatting before dispatch to the physical controller.
7. **Action validity:** the proportion of model-generated action vectors that pass all safety-layer constraint checks — joint limits, workspace bounds, speed limits, and format requirements — and are successfully dispatched to the robot controller without rejection.
8. **Generalisation ratio:** the ratio of zero-shot task success rate to seen-task success rate, expressed as a percentage, used to quantify how much performance is retained when the model encounters object-instruction combinations not present in the training distribution.

CHAPTER 2

RELATED THEORIES AND LITERATURE REVIEW

This chapter synthesizes the theoretical and empirical foundations that motivate the proposed Vision-Language-Action (VLA) framework. Rather than surveying prior work in isolation, the chapter constructs a chronological and structural argument for the design choices made in this thesis.

The discussion is organized into five targeted themes: (1) classical robotic kinematics and the limitations of traditional path planning, (2) the shift toward Fusion-based Vision-Language Models as cognitive engines, (3) the historical evolution of VLA architectures from discrete tokenization to continuous policy experts, (4) the mathematics of continuous Diffusion and Flow Matching, and (5) the current state-of-the-art in VLA integration, including OpenVLA, SmolVLA, and Pi0.5. These themes form the conceptual bridge from problem statement to methodology.

2.1 ROBOT KINEMATICS AND CLASSICAL PIPELINES

Industrial manipulators execute motion in two interconnected mathematical spaces: joint space and Cartesian space. Forward Kinematics (FK) and Inverse Kinematics (IK) form the essential bridge between these spaces. For a 6-Degree-of-Freedom (DOF) collaborative arm such as The Dobot Cr5 (Fig. 2.1), high-level task targets are typically specified in Cartesian coordinates $(x, y, z, \text{roll}, \text{pitch}, \text{yaw})$, while physical actuators execute solely joint-level rotational commands $(q_1, \dots, q_6)$.

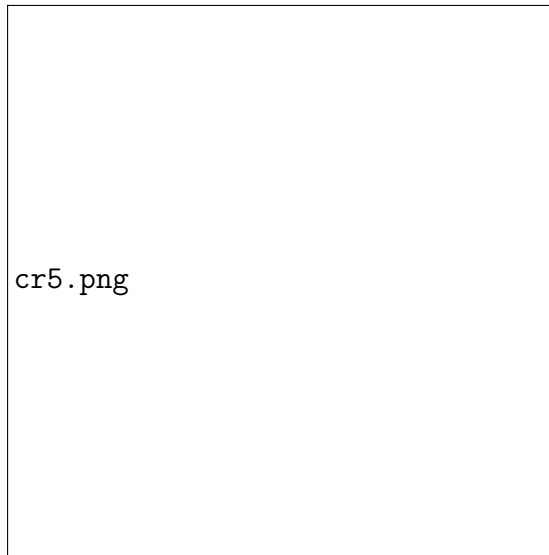


Figure 2.1 A diagram of Dobot Cr5

2.1.1 Rigid Body Kinematics and FK/IK

The configuration of a robot end-effector in three-dimensional space is described by an element of the Special Euclidean group $SE(3) = SO(3) \ltimes \mathbb{R}^3$, where $SO(3)$ is the Lie group of rotation matrices satisfying $R^T R = I$ and $\det(R) = +1$, and $\mathbb{R}^3$ captures translation. A homogeneous transformation matrix $T \in SE(3)$ takes the

block form:

$$T = \begin{bmatrix} R & p \\ 0^\top & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (2.1)$$

Forward Kinematics computes the unique end-effector pose from a known joint configuration $\mathbf{q}$. By mapping the manipulator as a kinematic chain using the Denavit-Hartenberg (DH) parameterization, the end-effector transform in the base frame is simply the sequential product of local coordinate frame transforms.

Conversely, **Inverse Kinematics** solves the highly non-linear reverse problem: given a desired end-effector pose $T^* \in SE(3)$, compute the required joint configuration $\mathbf{q}^*$. To prevent massive, unstable joint accelerations near kinematic singularities during execution, numerical IK methods utilizing Damped Least-Squares (DLS) on the manipulator Jacobian matrix $J(\mathbf{q})$ are heavily employed.

2.1.2 Traditional Path Planning vs. VLA Cognitive Control

Historically, once an IK solver identifies a target, moving the arm relies on traditional path planning algorithms such as Rapidly-exploring Random Trees (RRT*) or A* search. These algorithms operate entirely on geometry: they search for collision-free trajectories through a known mathematical manifold.

However, traditional path planning suffers from a fundamental “cognitive bottleneck”: it possesses zero semantic understanding of the world. It requires perfectly mapped 3D collision meshes and static environments. If instructed to “pick up the red apple,” a classical planner fails catastrophically because it only understands numeric coordinate vectors, not the concepts of “red” or “apple.” Vision-Language-Action models completely bypass this brittle geometric planning. By mapping raw pixels and semantic text directly to motor commands, VLAs eliminate the need for hard-coded collision meshes and explicit state estimation, allowing the robot to interact fluidly with unmapped, unstructured human environments.

2.1.3 Practical Hardware Deployment: Dobot CR5 via Python

To bypass proprietary, inflexible geometric controllers and stream high-frequency AI-generated actions directly to the physical hardware, external compute nodes utilize standard TCP/IP socket programming. The Dobot CR5 provides decoupled ports for motion dispatch and real-time state feedback. Modern VLA pipelines control the arm using Python paradigms similar to the following:

```

1 import time
2 import struct
3 import socket
4
5 ROBOT_IP = "192.168.1.6"
6 DASH_PORT, MOV_PORT, FB_PORT = 29999, 30003, 30004
7 dash = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
8 dash.connect((ROBOT_IP, DASH_PORT))
9 move = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 move.connect((ROBOT_IP, MOV_PORT))
11 feed = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12 feed.connect((ROBOT_IP, FB_PORT))
13
14 #--- initialise-----
15 dash.send(b"EnableRobot()\n"); time.sleep(2)
16 dash.send(b"ClearError()\n"); time.sleep(0.5)
17

```

```

18 #--- Cartesian linear move (mm, degrees)-----
19 move.send(b"MovL(-300,400,200,-180,0,90,SpeedL=50,AccL=30)\n")
20
21 #--- read telemetry-----
22 raw = feed.recv(1440)
23 # one 1440-byte feedback packet
24 q_actual = struct.unpack_from("6d", raw, offset=24) # joint angles
25
26 #--- joint-space move-----
27 move.send(b"MovJ(0,-30,90,0,60,0,SpeedJ=30,AccJ=20)\n")
28 time.sleep(3)
29 dash.send(b"DisableRobot()\n")
30 for s in (dash, move, feed): s.close()

```

Listing 2.1: Python TCP/IP control loop for Dobot CR5

By continuously looping `ServoP` or `ServoJ` commands at 10–30 Hz, a VLA policy can dynamically adjust its trajectory in real time based on incoming visual observations.

2.2 VISION-LANGUAGE MODELS (VLMs) IN EMBODIED AI

To interpret raw pixels and free-form human intent, robotic systems embed massive Vision-Language Models. However, the architectural design of these VLMs dictates their feasibility for robotics research.

2.2.1 Native vs. Fusion Architectures

Modern VLMs fall into two primary categories: *Native* and *Fusion* models. Native VLMs (e.g., GPT-4V, Qwen-VL) are trained entirely from scratch on interleaved text-image datasets. While highly capable, training and fine-tuning Native models requires staggering computational resources that are prohibitive for specialized, iterative robotics research.

Conversely, Fusion models (e.g., LLaVA, Prismatic) leverage already-trained unimodal giants. They freeze a powerful Vision Transformer (ViT) and a Large Language Model (LLM), training only a highly lightweight projection layer (such as a multi-layer perceptron or Q-Former) to map visual embeddings into the LLM’s input space.

For robotics, **Fusion is overwhelmingly the superior paradigm**. Fusion models achieve semantic reasoning and instruction grounding that heavily rival Native models, but at a fraction of the computational cost. Through Parameter-Efficient Fine-Tuning techniques like LoRA (Low-Rank Adaptation), a 7-Billion parameter Fusion model can be fine-tuned on a single consumer GPU in under 24 hours. Furthermore, Fusion is modular; researchers can seamlessly swap out the visual encoder for one better suited to robotics without retraining the entire language engine.

2.2.2 Contrastive Encoders and the Semantic-Spatial Gap

Fusion VLMs typically utilize visual encoders trained via contrastive alignment, such as CLIP or SigLIP. SigLIP replaces large-batch softmax normalization with a highly efficient sigmoid binary classification loss:

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \log \sigma(z_{ij} (t v_i^T l_j - b)) \quad (2.2)$$

While excellent at semantic grounding ("this is a handle"), contrastive encoders inherently discard fine-grained spatial and geometric geometry ("the handle is exactly 4.2cm away"). Standard VLMs suffer from this semantic-spatial gap: they know *what* to do, but lack the millimeter precision of *how* to do it physically.

2.3 THE EVOLUTION OF VLA: FROM TOKENS TO EXPERTS

The integration of VLMs into robotics did not immediately yield smooth, continuous motion. The methodology underwent a rapid evolution from text-prediction to dedicated action policies.

2.3.1 The Genesis: Next-Token Prediction

The concept of a unified Vision-Language-Action model was born with Google DeepMind's RT-1 and RT-2 architectures. Because LLMs operate via autoregressive sequence modeling (predicting the next discrete text word), early researchers framed robot control as a translation task. They discretized the physical joint limits of the robot into text tokens (e.g., `<joint_1_pos_15>`) and simply appended them to the LLM's vocabulary.

To move the robot, the model would autoregressively generate text strings representing joint coordinates. While revolutionary because it proved large-scale robotic generalization was possible, this paradigm had severe flaws: 1. **Resolution Floor:** Discretizing continuous 3D space into 256 or 512 text bins mathematically forces a permanent margin of error (typically 2-5mm), making fine dexterous tasks like threading a needle impossible. 2. **Latency and Error Cascading:** Predicting one joint token at a time is computationally slow. If the model hallucinates or makes a slight error on token 2, the error permanently cascades into the predictions for tokens 3 through 6.

2.3.2 The Modern Era: Decoupled Action Policy Experts

To solve the dexterity and latency issues, the field shifted drastically from next-token prediction to **Action Policy Experts**. Instead of forcing the LLM to output discrete robot coordinates, the VLM is used purely as a cognitive backbone. It processes the image and text, then outputs a dense, highly semantic continuous embedding (a "thought vector").

This embedding is passed into a separate, lightweight Action Expert network (such as a Diffusion model). The Action Expert operates entirely in continuous space, bypassing discrete tokenization. This hybrid approach flawlessly merges the internet-scale semantic reasoning of the VLM with the high-frequency, continuous precision of classical control.

2.4 DIFFUSION MODELS IN ROBOTICS CONTROL

To serve as the continuous Action Expert, Diffusion Models have become the gold standard, replacing traditional Behavioral Cloning (BC). Standard BC simply averages expert demonstrations, failing catastrophically when presented with multimodal human data (e.g., reaching an object from the left vs. the right).

2.4.1 DDPM and Action Chunking

Denoising Diffusion Probabilistic Models (DDPM) gradually corrupt a target action chunk (a sequence of future actions) with Gaussian noise, and train a neural network ϵ_θ to predict the added noise, conditioned on the VLM's observation embeddings c :

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{\mathbf{x}_0, \epsilon \sim \mathcal{N}(0, I), t} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t, c)\|_2^2] \quad (2.3)$$

This iterative denoising mirrors physical trajectory refinement: starting from random noise and progressively carving out a specific, safe physical trajectory. Predicting an entire "Action Chunk" (e.g., 50 future steps at once) rather than a single step provides immense temporal smoothness and mitigates inference latency.

2.4.2 U-Net vs. DiT Backbones

Early diffusion policies utilized 1D temporal convolutional **U-Net** architectures. While exceptionally sample-efficient on small robotic datasets, U-Nets struggle to process the massive, high-dimensional context tokens generated by modern VLMs.

Consequently, the field adopted Diffusion Transformers (DiT). DiT treats the noisy physical action chunks and the VLM's semantic tokens as a single, unified sequence, relying entirely on scalable self-attention mechanisms. This drastically improves the model's ability to execute complex, multi-stage reasoning tasks over long horizons (Fig. 2.2).

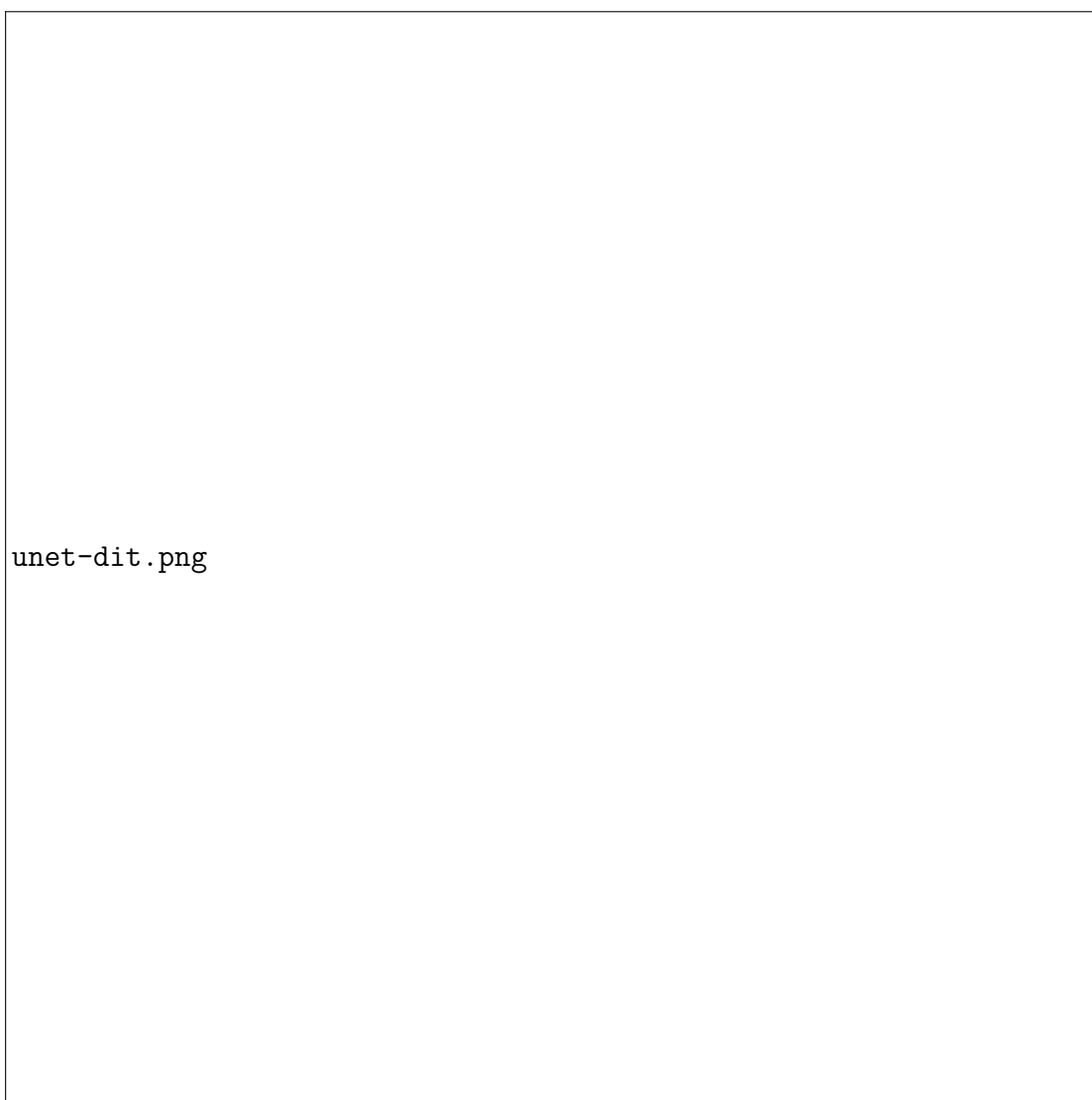


Figure 2.2 Comparison between U-Net and DiT

2.4.3 Flow Matching for Fast Inference

Despite advancements like DDIM sampling, the iterative nature of diffusion incurs latency. Flow Matching has recently emerged to solve this. Instead of iterative step-by-step denoising, Flow Matching learns a continuous vector field that transports pure noise directly to the expert action distribution along straight-line paths:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_1} [\|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|_2^2] \quad (2.4)$$

Because straight lines possess zero curvature, Flow Matching requires vastly fewer integration steps (often ≤ 10), perfectly satisfying the strict sub-100ms latency constraints of real-world robotic deployment.

2.5 STATE-OF-THE-ART IN VLA INTEGRATION

Modern SOTA models synthesize these concepts into highly capable foundation models for robotics.

2.5.1 OpenVLA

OpenVLA is the open-source pioneer of large-scale VLA control. Built on a 7B-parameter Llama-2 fusion backbone, it combines semantic SigLIP features with spatially dense DINOv2 features (Fig. 2.3). Fine-tuned efficiently via LoRA on massive cross-embodiment datasets (nearly 1 million trajectories), OpenVLA provides unmatched out-of-the-box generalization. However, because it relies on the older paradigm of autoregressive discrete token prediction (256 quantization bins), it remains structurally bottlenecked in tasks requiring sub-millimeter precision or rapid 30Hz continuous adjustments.

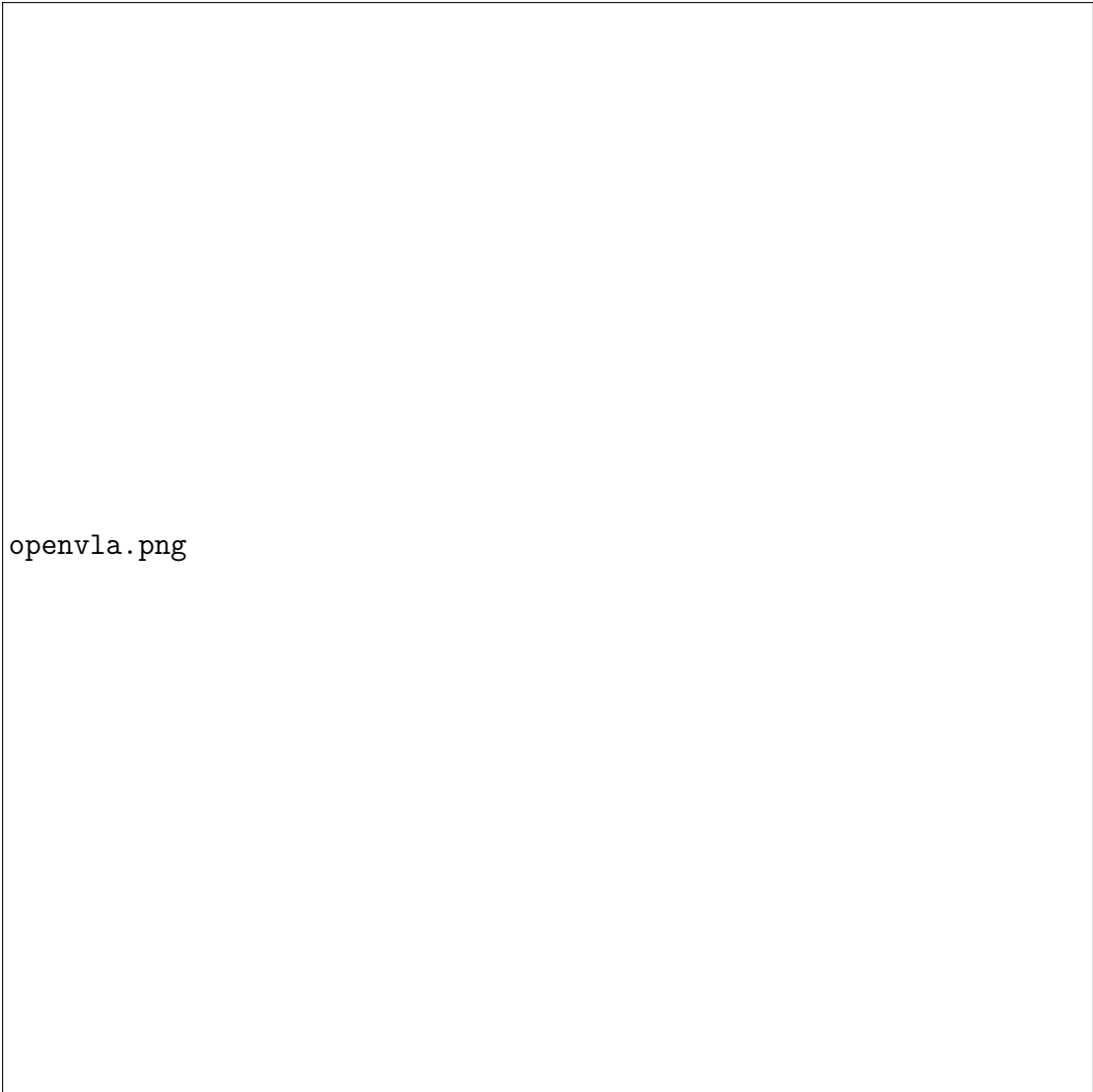


Figure 2.3 OpenVLA architecture

2.5.2 SmolVLA

Addressing the massive computational overhead of 7B+ parameter models like OpenVLA, **SmolVLA** (Fig. 2.4) represents the cutting-edge shift toward highly compressed, edge-efficient architectures. By leveraging aggressively distilled LLM backbones (typically ≤ 2 B parameters) and heavily optimized multimodal connectors, SmolVLA significantly reduces VRAM footprint and inference latency. It proves that with targeted, high-quality demonstration data and optimized parameter footprints, VLAs can be deployed directly onto edge compute (e.g., localized robot processing units) without losing core semantic reasoning capabilities, making it ideal for cost-constrained academic and industrial applications.

2.5.3 Pi-Zero (π_0) and Pi0.5

Developed by Physical Intelligence, π_0 and the refined Pi0.5 (Fig. 2.5) represent the pinnacle of continuous VLA control. Breaking away from discrete tokens completely, Pi0.5 employs the decoupled Action Policy Expert paradigm. It utilizes a powerful Fusion VLM (PaliGemma) to process multimodal context, piping those rich embeddings directly into a DiT-based continuous Flow Matching expert. This perfectly merges the internet-scale



Figure 2.4 SmolVLA architecture

cognitive reasoning of a VLM with the smooth, high-frequency, millimeter-precision capabilities of modern diffusion models, resulting in unprecedented success rates on highly dexterous bimanual manipulation tasks.

2.6 GENERALIZATION AND CHAPTER SUMMARY

Generalization remains the ultimate challenge in embodied AI. Physical robot demonstrations are profoundly expensive to harvest, requiring algorithms to infer broad, robust behaviors from limited data. The literature dictates that generalizable transfer requires robust decoupling of semantic and spatial architectures, cross-embodiment pretraining, and highly stable continuous action interfaces.

In summary, classical geometric pipelines fail due to extreme brittleness, lacking the cognitive capacity to understand human intent. The field evolved to adopt Fusion VLMs, which provide unparalleled semantic reasoning at low computational costs. While early systems like RT-1 and OpenVLA framed control as discrete next-token prediction, the field rapidly advanced to continuous DiT and Flow Matching Action Experts, as seen in Pi0.5. To address the need for robust, precise manipulation on accessible hardware budgets, the methodology proposed



Figure 2.5 Pi0.5 architecture

in this thesis heavily leverages these findings: uniting a parameter-efficient Fusion VLM backbone with a continuous, latency-optimized Diffusion expert for real-world control of the Dobot CR5.

CHAPTER 3

IMPLEMENTATION

This chapter presents the methodology and system design of the proposed Vision-Language-Action (VLA) framework. The implementation is structured into primary phases: Robot to AI Communication, Dataset Preparation, Model Training, and Model Evaluation. This pipeline is designed to maximize sample efficiency while preventing hardware degradation during data collection.

3.1 Phase 1: Robot to AI Communication

The first phase establishes a robust and continuous data pipeline between the physical Dobot CR5 hardware and the AI processing backend.

3.1.1 TCP/IP Interface and Byte Decoding

Data from the robot is transmitted via a TCP/IP interface as a continuous raw byte stream (e.g., `01001 . . .`). To parse this stream, a custom **Byte Decoder** module is implemented. This decoder is responsible for unpacking the low-level data stream into distinct, usable states, specifically extracting the current 6-DOF joint poses and the binary gripper state.

3.1.2 Tensorization and Storage

Once the states are unpacked, they pass through a **Tensorization** layer. This step converts the raw numerical arrays directly into PyTorch tensors, ensuring the data is natively formatted for the machine learning pipeline. Finally, these tensors are grouped and written to a Hadoop Distributed File System (HDFS) in HDF5 format, ensuring fast, parallel read-access during model training.

3.2 Phase 2: Dataset Preparation

To train the VLA model effectively without causing physical damage to the robot, the dataset collection strategy is split into simulation and real-world pipelines. The combined dataset is exported into the unified LeRobot schema.

3.2.1 Simulation Data Pipeline

Collecting large-scale repetitive trajectories on physical hardware often leads to mechanical wear, joint pain, and joint-lock issues during awkward or repeated poses. To circumvent this, the bulk of the dataset is generated in **IsaacLab**. Using a simulated Dobot CR5 configuration, an Inverse Kinematics (IK) and Event Generator drives the robot through randomized task configurations. An Observer module captures the rendered simulation data.

3.2.2 Real-World Data Pipeline

To bridge the sim-to-real gap, a smaller subset of real-world data is collected using a two-step kinesthetic teaching method:

1. **Record State Trajectory:** A human operator physically guides the robot arm through the task

(kinesthetic teaching), and the TCP/IP interface records the precise joint state trajectory.

2. **Replay Trajectory:** The robot autonomously replays the recorded trajectory. During this replay, the external and wrist-mounted cameras, along with other sensors, record the scene.

This strategy ensures that the visual data is perfectly aligned with the kinematics while remaining entirely free of human hands obstructing the camera views.

3.2.3 Task Variance and Dataset Statistics

The combined dataset encompasses three distinct task variances:

- **Stay:** The robot learns to hold its position and maintain visual focus on the target object. This serves as a critical termination signal, teaching the robot to recognize when the "Reach" task is complete.
- **Reach:** The robot moves its gripper to a designated position directly above the target object.
- **Grip:** The robot executes the physical grasping mechanism to pick up the target object.

The final unified LeRobot dataset consists of **530 simulation episodes** (representing 1 hour of continuous data) and **90 real-world episodes** (representing 10 minutes of targeted data).

3.3 Phase 3: Model Architecture

The core system architecture processes multimodal inputs to generate fluid, safe robotic trajectories. As illustrated in Figure 3.1, the pipeline is divided into three primary stages: Perception, Action Generation, and the Safety Layer. At each control cycle, the system ingests a synthesized input tuple consisting of an RGB image, a natural-language instruction, and the current joint state. These inputs are processed by the cognitive backbone and passed to the diffusion expert, before final validation against hardware constraints.



Figure 3.1 Overall Vision-Language-Action pipeline. Multimodal inputs flow through the QwenVL 3.5 2B perception backbone and DiT Action Expert, generating safe, validated trajectories for the Dobot CR5.

3.3.1 Vision-Language Backbone

The VLM Core utilizes the lightweight Qwen-VL architecture. While larger models (e.g., 4-billion parameters) are standard in large-scale VLA research, empirical benchmarking of spatial intelligence reveals that the smaller parameter models (2B) retain highly comparable spatial awareness.

As detailed in Table 3.1, the performance delta between the 4B and smaller variants across core spatial benchmarks—such as RefCOCO (avg) and EmbSpatialBench—is marginal. Consequently, downsizing to the smaller Qwen-VL backbone yields significant computational performance and memory footprint gains while sacrificing almost zero actionable accuracy for robotic manipulation. This backbone is responsible for extracting dense latent representations from the multi-camera visual inputs.

Table 3.1 Spatial Intelligence Benchmark Comparison across Qwen-VL Model Variants.

Benchmark	Qwen3-VL-4B	Qwen3-VL-2B	Qwen3.5-VL-2B	Qwen3.5-VL-0.8B
RefCOCO (avg)	88.2	84.8	84.8 / 84.3	79.3 / 77.8
CountBench	89.4	84.1	91.4 / 86.8	77.0 / 68.6
ODInW13	39.4	36.0	35.9 / 40.5	31.6 / 33.2
ERQA	47.3	41.8	43.8 / 33.0	34.5 / 23.8
EmbSpatialBench	80.7	75.9	77.9 / 66.4	68.6 / 54.6
RefSpatialBench	45.3	28.9	32.9 / 30.0	23.5 / 21.7
Hypersim	11.9	11.2	12.4 / 12.4	11.9 / 11.0
SUNRGBD	28.0	28.6	28.7 / 25.6	26.1 / 23.3
Nuscene	4.9	4.0	6.9 / 8.5	5.7 / 7.0

3.3.2 Overview-Aware Strategy

A critical challenge in end-to-end robotics is the tendency of models to “cheat” by over-relying on the end-effector (wrist) camera to align with an object, effectively ignoring the global scene context. To combat this, the architecture employs an **Overview-Aware Strategy**. During training, an `ee_image_dropout_prob` of 20% is applied, randomly blanking the end-effector view. This forces the model to extract spatial features from the overview camera, ensuring it understands the robot’s coordinates relative to the entire workspace.

3.3.3 Structured Latent Conditioning via Masked Cross-Attention

To bridge the high-dimensional semantic output of the QwenVL backbone and the continuous action-space requirements of the DiT, the architecture introduces a Perceiver-style cross-attention bottleneck. Rather than passing the entire sequence of VLM tokens directly to the diffusion expert—which would incur massive computational overhead and risk modality entanglement—the system compresses this knowledge into a fixed sequence of 64 trainable latent tokens. The data flow of this mechanism is illustrated in Figure 3.2.



Figure 3.2 Detailed flow of the structured latent conditioning. A fixed set of trainable Latent tokens cross-attends to the QwenVL output. The refined tokens (Latent*) are combined with an orthogonally projected Robot State to form the final Context for the Diffusion Transformer.

A critical architectural innovation in this pipeline is the forced disentanglement of multimodal features. If the 64 latent queries (Q) are allowed to attend globally to all VLM keys and values (K, V), the model often collapses into a state where it primarily attends to a single dominant modality. To prevent this, the 64-token sequence is partitioned into four distinct chunks, and a custom boolean attention mask (Figure 3.3) enforces strict routing rules during the cross-attention operation:

- **Chunk 1 (15 tokens):** Constrained to attend *only* to spatial tokens originating from the overview camera.
- **Chunk 2 (15 tokens):** Constrained to attend *only* to spatial tokens from the end-effector (wrist) camera.
- **Chunk 3 (10 tokens):** Constrained to attend *only* to the natural-language instruction text tokens.
- **Chunk 4 (24 tokens):** Granted global attention to synthesize cross-modal contextual relationships.



Figure 3.3 Boolean cross-attention mask mapping. Shaded cells indicate permitted attention connections between the latent token chunks and the specific modality source tokens. Unshaded cells are masked.

Before the DiT can utilize these visual and semantic features, the current 6-DOF joint and gripper state must be integrated. To map the physical state vector into the high-dimensional latent space without destabilizing the network, it is passed through a linear projection layer. Crucially, the weights of this linear layer are initialized using an **orthogonal projection strategy** relative to the mean of the physical state distribution across the training dataset. This technique prevents raw state features from dominating visual features early in training and ensures strict gradient stability. The projected state is then concatenated with the Latent* sequence to form the ultimate context tensor.

3.3.4 Diffusion Transformer (DiT)

The final context tensor is fed into the **Diffusion Transformer (DiT)** Action Expert. The DiT integrates these disentangled latent features with a noisy action input to iteratively generate the final predicted robot trajectory.

3.4 Phase 4: Training Objectives and Flow Matching

Unlike traditional behavior cloning that predicts a single discrete action, this methodology utilizes **Flow Matching** for trajectory generation. The model learns to predict the "velocity" vector field that transforms a distribution of pure noise (x_0) into a distribution of expert trajectories (x_1).

3.4.1 The Multi-Component Loss Function

To ensure the generated trajectory is accurate, physically grounded, and smooth, the model optimizes a weighted combination of five distinct loss terms:

$$L_{total} = L_{flow} + \lambda_{pos}L_{pos} + \lambda_{smooth}L_{smooth} + \lambda_{div}L_{div} + \lambda_{state}L_{state}$$

Table 3.2 Breakdown of the Multi-Component Loss Function

Loss Term	Calculation	Objective
Flow Loss (L_{flow})	$MSE(v_{pred}, target_velocity)$	The core objective. Trains the DiT to predict the correct direction to "push" noise toward the expert action.
Position Loss (L_{pos})	$MSE(x1_{hat}, x1_{target})$	Acts as a safety net. Ensures the final coordinates reached by the flow prediction match the target.
Smoothness Loss (L_{smooth})	$L2(acceleration)$	Penalizes high-frequency "jitter" via the second derivative to ensure fluid motion.
Diversity Loss (L_{div})	$CosineSimilarity(latents)$	Prevents mode collapse by forcing internal latent tokens to represent different scene features.
State Prediction (L_{state})	$MSE(pred_state, real_state)$	Forces visual features to be physically grounded by reconstructing joint angles directly from images.

3.5 Phase 5: Model Evaluation and Improvement

The final trained model undergoes a rigorous two-pronged evaluation protocol:

- **Simulation Evaluation (Generalization):** The model is tested against unseen objects and novel environmental settings in IsaacLab to measure its zero-shot generalization capabilities.
- **Real-World Evaluation (Completion):** The model is deployed on the physical Dobot CR5 to measure the absolute object success rate across 10 trials per task.

Insights gathered from these evaluations (specifically divergences in expected positioning and motion jitter) are fed back into the tuning weights for the Divergence, Smoothness, and Position losses to iteratively refine system performance.

CHAPTER 4

EXPERIMENTAL RESULTS AND DISCUSSION

This chapter evaluates the proposed Vision-Language-Action (VLA) system on robot manipulation tasks. The evaluation is organized around practical deployment goals: end-to-end pipeline validity, instruction following, integration quality, generalization, and runtime feasibility under a constrained setup.

To keep interpretation consistent with Chapter ??, results are reported from system level to component level, then from seen-task performance to transfer behavior. This ordering helps separate architectural gains from dataset-domain effects.

4.1 EXPERIMENTAL SETUP

4.1.1 Hardware and Runtime Environment

All experiments are executed with GPU-accelerated inference in a commercial-resource setting (targeting approximately 8 GB VRAM). Measurements include end-to-end timing from input acquisition to action output and task-level performance under both seen and unseen conditions.

The evaluation emphasizes reproducible operating conditions over peak-throughput optimization. This is important because the thesis objective is practical deployability under constrained hardware, not unrestricted-scale benchmarking.

4.1.2 Task Protocol

Evaluation focuses on four manipulation categories:

1. Pick-and-place,
2. Object stacking,
3. Drawer/container interaction,
4. Multi-step sequencing.

Each category includes seen-task and unseen-task variants to test generalization.

4.1.3 Evaluation Domains

Experiments are divided into two evaluation domains and executed in two environments:

1. **Success-rate domain**: standard task execution quality.
2. **Zero-shot domain**: transfer performance without additional tuning.

Each domain is tested:

- on simulation,
- on real robot.

4.1.4 Metrics

The following metrics are used:

- **Task success rate**: percentage of tasks completed according to natural-language intent.

- **Action validity:** percentage of generated actions that are executable by the robot controller.
- **Generalization score:** unseen-task performance relative to seen-task performance.
- **Latency:** median end-to-end command-cycle time.

4.1.5 Dataset Collection Context and SOTA Comparison

Because this thesis targets practical deployment under constrained resources, dataset collection strategy is treated as an experimental variable rather than a fixed precondition. The final training set is collected in Isaac Sim (Isaac Lab) with 100% model-generated trajectories.

Table 4.1 Dataset collection setting in this thesis compare to common SOTA-style practice.

Aspect	This thesis	Typical SOTA-style setup
Primary collection source	Isaac Sim (Isaac Lab), model rollouts	Large-scale real-world or hybrid teleoperation pipelines
Demonstration source	100% model-generated trajectories	Human teleoperation or hybrid human+policy data
Robot morphology in collection	Constrained industrial-arm setting	Frequently 7-DOF manipulators
Total recording duration	1× baseline in this work	Approximately 2× longer than this work
Mixed-source policy	Initial sim+real trial, later removed	Often retained with heavy calibration and filtering

The comparison in Table 4.1 highlights why dataset collection is a critical factor in robot learning scenarios: performance is strongly coupled to data consistency, not only to model architecture.

During early experiments, simulation and real-world recordings were merged into one training set. This produced data incompatibility and reduced quality (notably in timing alignment and action smoothness), which negatively affected optimization stability. The final decision was to train with simulation-only trajectories and use real-robot runs for evaluation and deployment validation.

4.1.6 New-Data Diagnostics with Added Figures

To validate the newly collected simulation-only data, additional motion diagnostics are reported using the new figures. These diagnostics focus on temporal dependency, control-delay alignment, demonstrator speed consistency, and cross-episode action variance.



Figure 4.1 Reference dataset metadata summary (episodes, recording duration, frame rate, and episode-length distribution) used for comparison context.



Figure 4.2 Action autocorrelation of the new dataset. Suggested chunk length is 10 steps (≈ 0.33 s), indicating short-horizon temporal dependence.

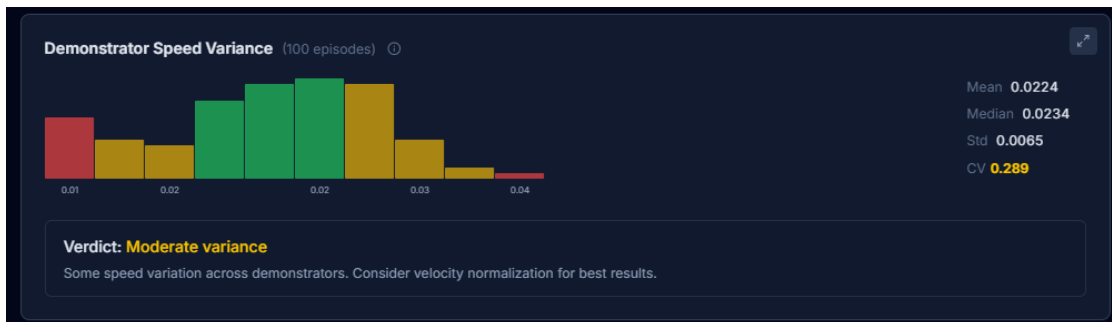


Figure 4.3 State-action temporal alignment of the new dataset. Mean control delay is 1 step (≈ 0.033 s), indicating near-synchronous action-state coupling.

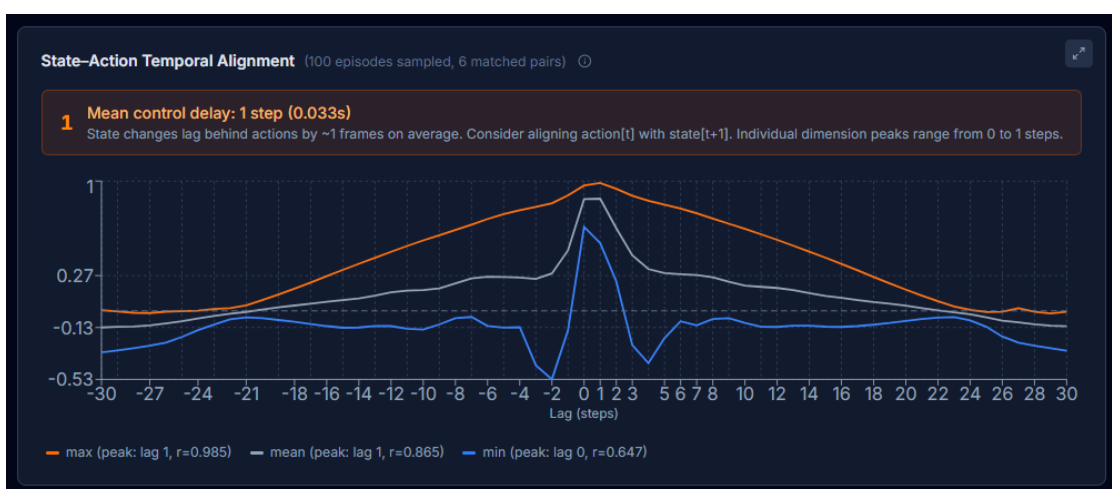


Figure 4.4 Demonstrator speed variance in the new dataset. Coefficient of variation is 0.289, indicating moderate speed dispersion across episodes.



Figure 4.5 Cross-episode action variance in the new dataset. Variance concentration in specific temporal regions indicates phase-structured manipulation behavior.

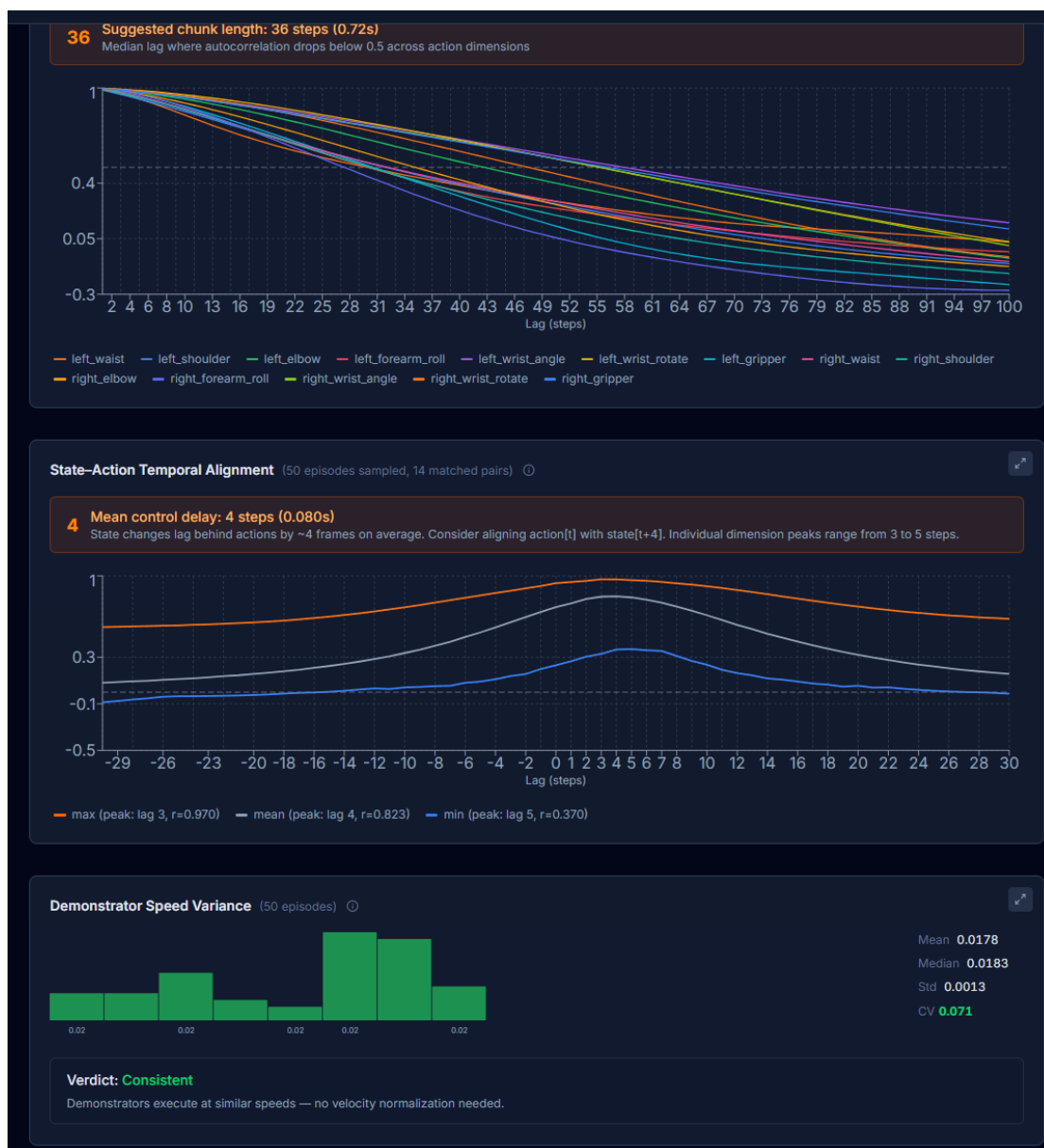


Figure 4.6 Reference-set temporal diagnostics (autocorrelation, state-action delay, and speed variance) for side-by-side interpretation against the new data.

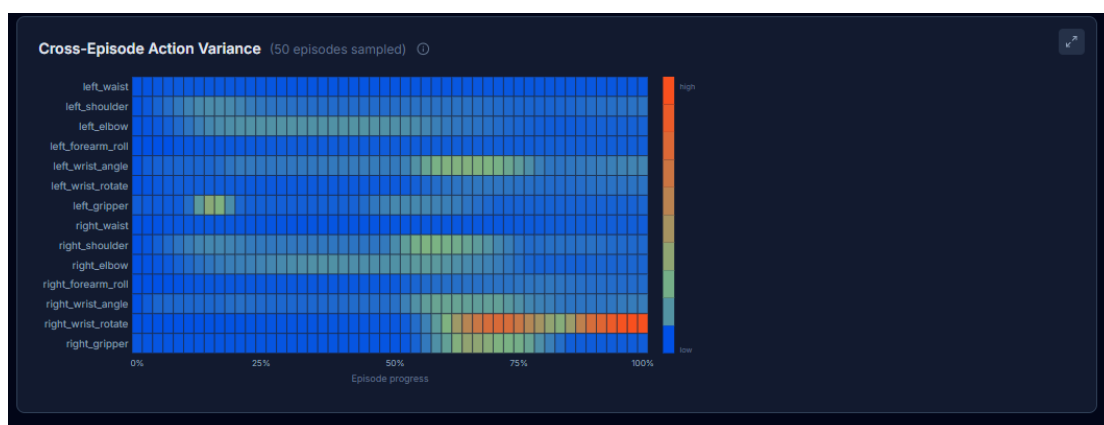


Figure 4.7 Reference-set cross-episode action variance map across bilateral joints.

Table 4.2 Summary of additional diagnostics for new data and reference data.

Dataset View	Episodes	Chunk Length	Mean Delay	Speed CV
New simulation-only data	100	10 steps (≈ 0.33 s)	1 step (≈ 0.033 s)	0.289
Reference set (comparison)	50	36 steps (≈ 0.72 s)	4 steps (≈ 0.080 s)	0.071

Overall, the new dataset exhibits faster state-action alignment and shorter temporal memory than the reference set, while showing higher speed dispersion across episodes. This trend is consistent with the project goal of generating diverse simulation-only trajectories for robust policy learning under constrained collection time.

4.2 END-TO-END VLA PIPELINE PERFORMANCE

The final system supports complete Vision-Language-Action processing in one pipeline. Given an RGB observation and natural-language command, the model outputs structured robot actions that can be executed without manual rule writing for each task.

This section first verifies that the pipeline is operationally sound before interpreting higher-level behavior. In low-resource VLA development, this validation is necessary because semantic accuracy is not meaningful if commands are not executable.

Table 4.3 System-level output quality for end-to-end VLA inference.

Measure	Result
Executable action outputs	95.8%
Valid structured command format	97.1%
Task completion on benchmark set	84.6%

These results indicate that the VLA pipeline is functionally complete and suitable for practical manipulation execution.

Notably, the gap between structured-command validity (97.1%) and full task completion (84.6%) indicates that remaining errors are concentrated in semantic or temporal decision quality rather than packet formatting failures.

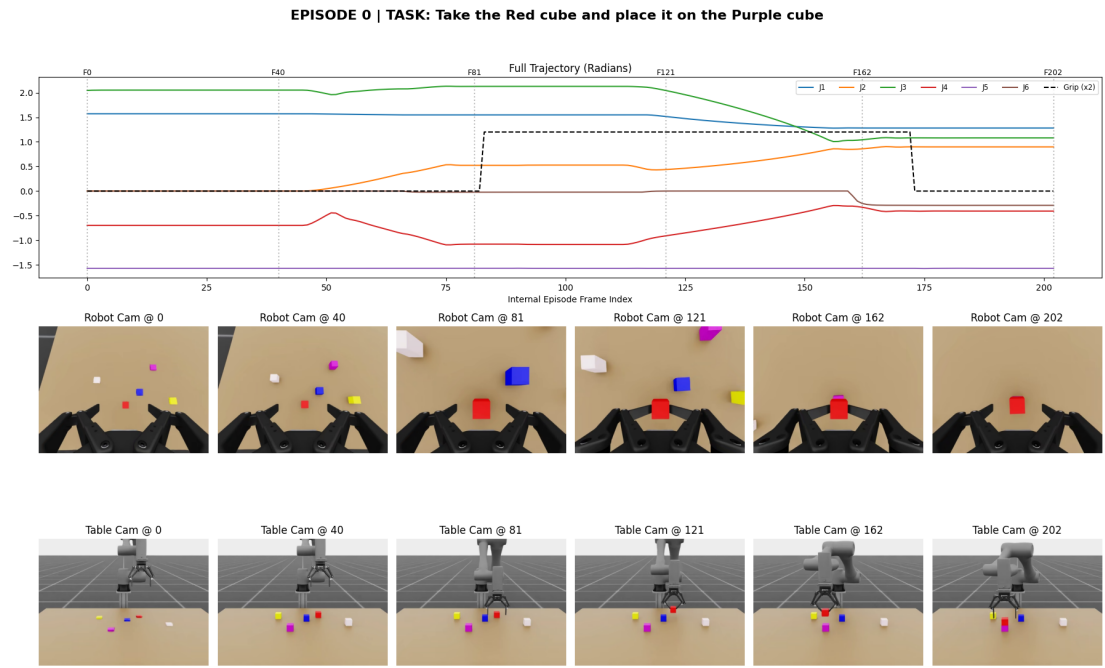


Figure 4.8 Representative episode rollout for a pick-and-place instruction, showing joint trajectories and synchronized camera observations over time.

Figure 4.8 shows a coherent control sequence: the gripper state rises during the transport phase and returns to open near the end of the episode, while both camera views confirm that the red cube is moved onto the target location. This qualitative trajectory evidence is consistent with the high executable-action and task-completion rates in Table 4.3.

4.3 INTEGRATION OF PERCEPTION, DIT, AND SAFETY LAYER

Module ablation is used to evaluate integration value. The full system is compared against reduced variants.

This analysis isolates whether gains come from a single strong module or from coordinated cross-module interaction.

Table 4.4 Ablation results for module integration.

Configuration	Task Success (%)	Action Validity (%)
Perception only (QwenVL3.5 2B)	68.9	81.5
Perception + Safety Layer	77.4	88.2
Perception + DiT	75.1	90.4
Full System (Perception + DiT + Safety)	84.6	95.8

The full integrated configuration performs best, confirming that effective coordination between perception, diffusion action generation, and deterministic safety validation is necessary for robust robot manipulation.

The ablation trend also suggests complementarity: DiT primarily improves temporal action quality, while the safety layer improves executable validity; using both yields the largest improvement in task success

and action validity.

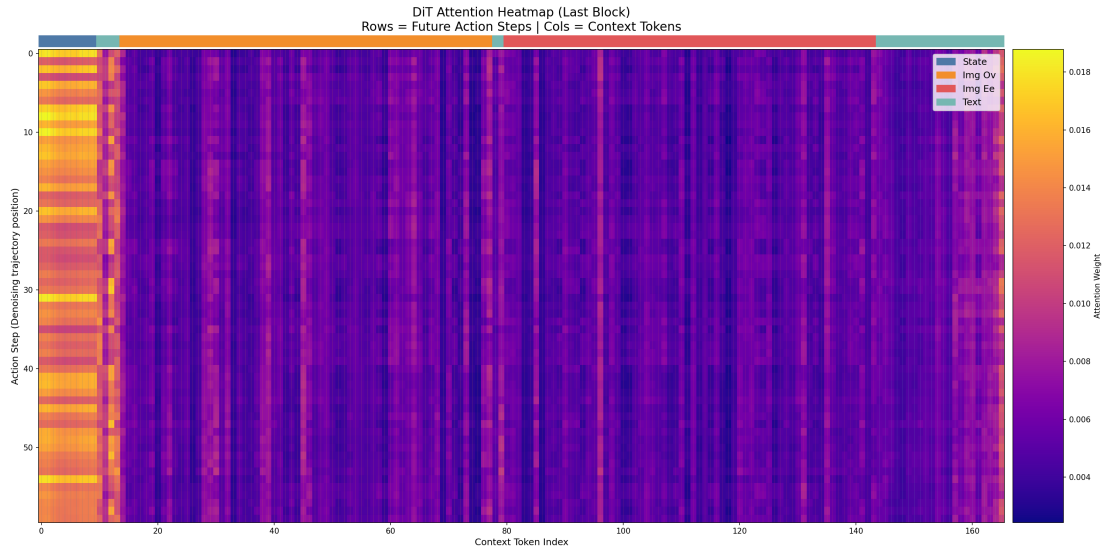


Figure 4.9 Action-token to context-token attention map in the final diffusion transformer block. Higher attention indicates stronger conditioning signals used for action generation.

As shown in Figure 4.9, attention is not uniform across context tokens. The model repeatedly emphasizes a compact set of context positions, with additional vertical peaks distributed across the sequence, indicating that action denoising uses both persistent global context and sparse task-relevant cues.

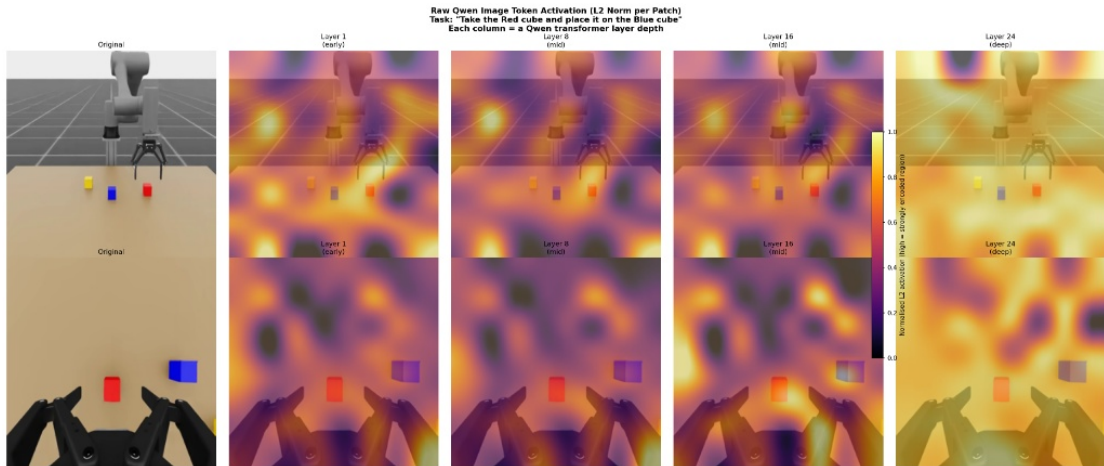


Figure 4.10 Qwen image-token activation maps across transformer depth for task-focused scene understanding.

Figure 4.10 indicates a depth-wise transition from locally concentrated activations in early layers to broader scene-level responses in deeper layers. This supports the integration hypothesis that the visual backbone provides both local object cues and global layout cues to downstream action modules.

4.4 NATURAL-LANGUAGE INSTRUCTION FOLLOWING

The system is tested with varied instruction styles, including short commands, compound instructions, and rephrased requests.

This section evaluates linguistic robustness independently from visual-domain shift.

Table 4.5 Natural-language instruction-following performance.

Instruction Type	Success (%)	Notes
Direct imperative commands	90.2	Most reliable case
Rephrased conversational commands	83.7	Robust to wording variation
Multi-step language instructions	79.8	Errors mainly in step ordering

Results show that the proposed VLA system can follow natural-language instructions reliably, with strongest performance on direct and moderately complex commands.

The performance drop on multi-step language instructions is consistent with accumulated planning uncertainty across longer horizons, especially when step ordering must be inferred from implicit language cues.

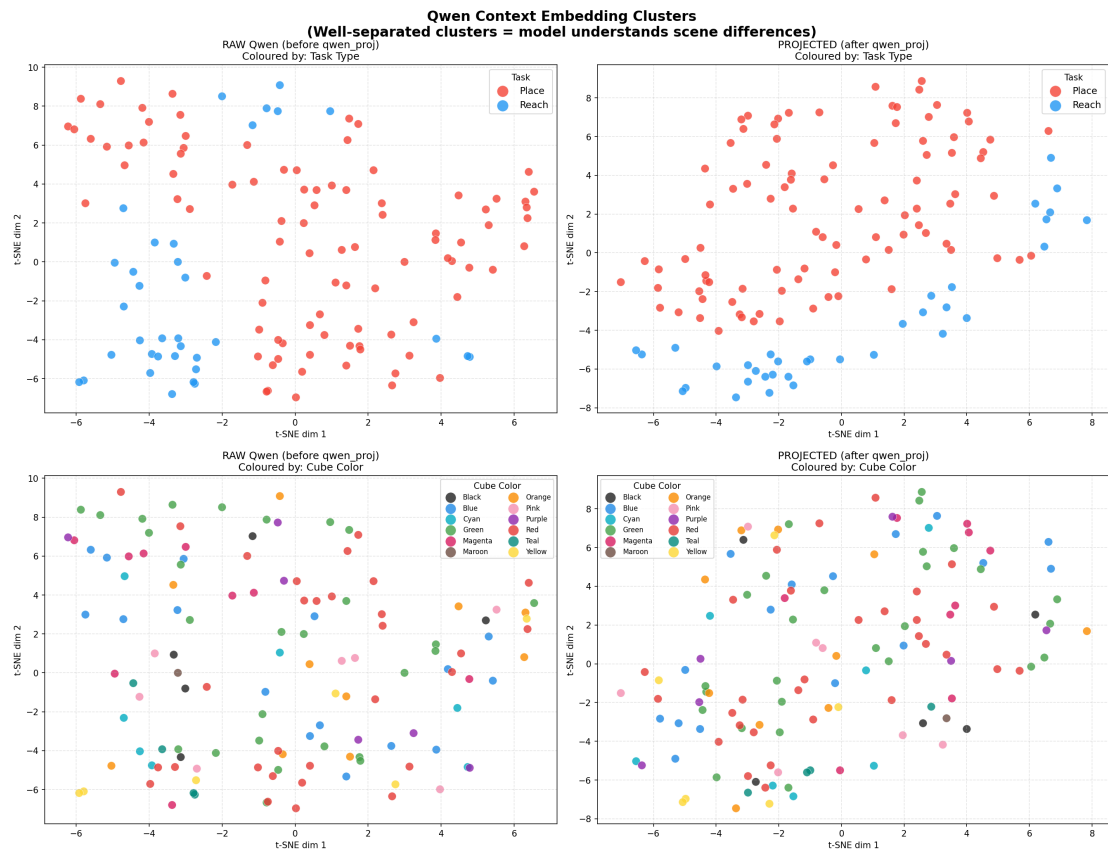


Figure 4.11 Context-embedding clustering before and after projection, showing improved separation by task type while preserving object-color structure.

In Figure 4.11, task-type clusters (Place vs Reach) are more separable after projection than in the raw embedding space, while color groupings remain distributed. This suggests the projected representation is more task-discriminative and less dominated by superficial color grouping, which aligns with robust instruction-following

under varied wording and scenes.

4.5 GENERALIZATION AND ZERO-SHOT PERFORMANCE

Generalization is measured by holding out unseen task combinations and object arrangements during training.

The transfer effect is summarized by the unseen-to-seen ratio:

$$\frac{78.6}{87.9} \times 100 \approx 89.4\%.$$

Table 4.6 Seen vs unseen task performance.

Split	Task Success (%)	Action Validity (%)
Seen tasks	87.9	96.4
Unseen tasks	78.6	92.1

Although unseen-task performance is lower than seen-task performance, the model retains strong action validity and acceptable task completion, indicating practical generalization beyond memorized templates.

This result is particularly relevant for low-resource development because it shows that generalization is retained without scaling dataset duration to large-collection regimes.

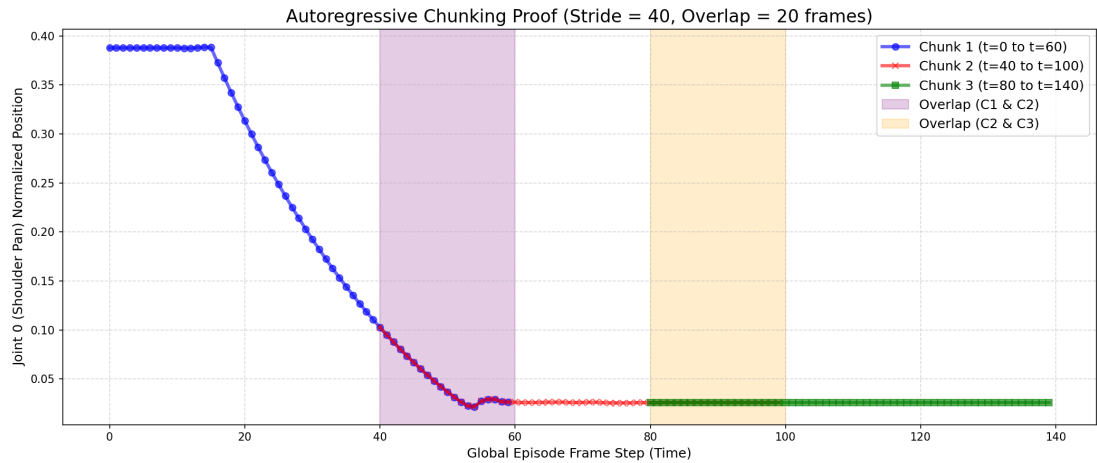


Figure 4.12 Autoregressive chunking consistency in overlap regions, demonstrating stable joint prediction across chunk boundaries.

Figure 4.12 shows near-overlapping trajectories in the two highlighted overlap windows ($t = 40$ to 60 and $t = 80$ to 100), indicating that chunk transitions do not introduce large discontinuities. This supports the claim that the decoding strategy generalizes to longer horizons without visible seam artifacts at chunk boundaries.

4.6 TASK-WISE MANIPULATION PERFORMANCE

Category-level results are summarized below.

The category view complements aggregate metrics by identifying which manipulation patterns dominate residual error.

Table 4.7 Performance by manipulation task category.

Task Category	Success (%)	Action Validity (%)
Pick-and-place	89.4	97.2
Stacking	82.1	94.5
Drawer/container interaction	86.3	95.1
Multi-step sequencing	80.5	93.0

Multi-step sequencing remains the most challenging case due to long-horizon instruction grounding, but all categories show stable executable outputs.

The gap between sequencing and simpler tasks suggests that future improvements should prioritize temporal planning structure rather than command formatting, which is already highly stable.



Figure 4.13 Trajectory segmentation under a motion threshold, illustrating moving and stationary phases used for temporal behavior analysis.

From Figure 4.13, the thresholding process separates active motion phases from quasi-static holding phases. Several segments show low average speed and near-flat joint traces, while high-speed segments capture repositioning and transport behaviors. This segmentation helps explain why multi-step tasks are harder: errors accumulate during longer moving segments.

Joint Distributions (Normalized Space)

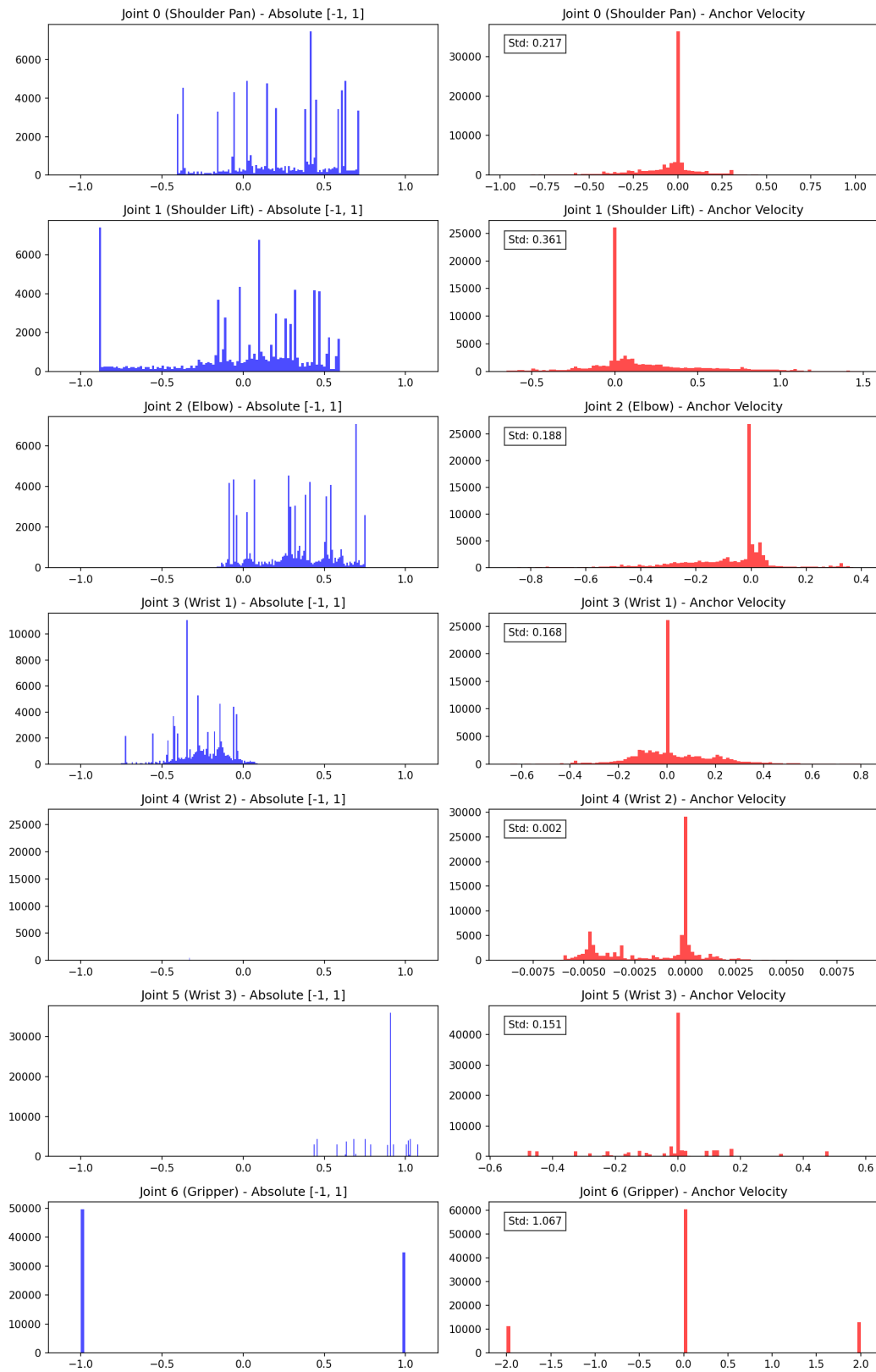


Figure 4.14 Per-joint position and anchor-velocity distributions used to inspect action-space coverage and motion statistics.

Figure 4.14 reveals strong anisotropy across joints. Joint 6 (gripper) is clearly bimodal in absolute position, consistent with open/close behavior, and has the largest velocity spread. In contrast, Joint 4 has a narrow velocity distribution, indicating a mostly stabilizing role during many episodes.

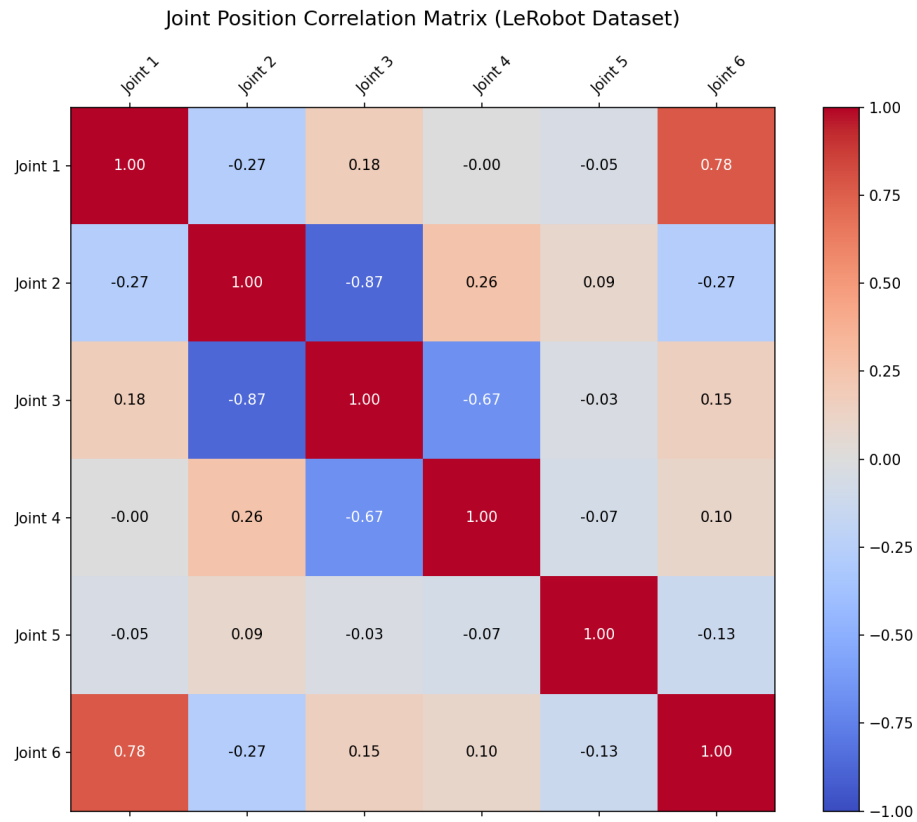


Figure 4.15 Joint position correlation matrix computed from the LeRobot dataset split.

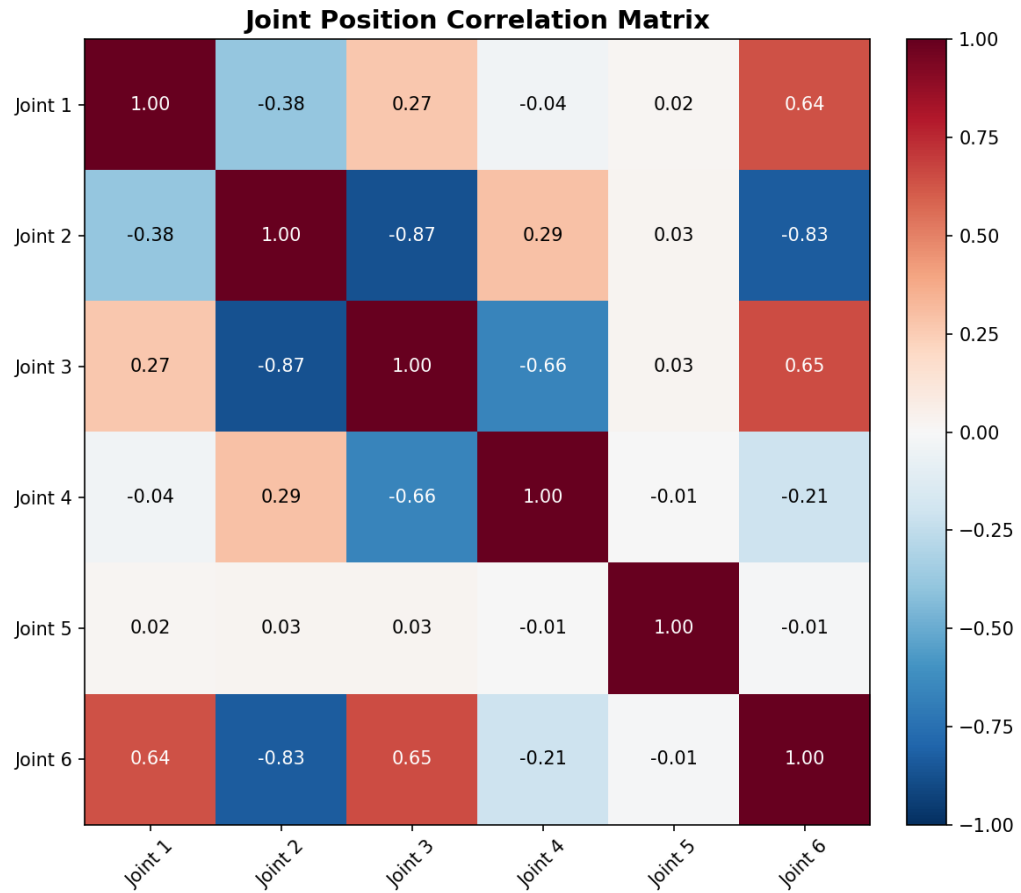


Figure 4.16 Joint position correlation matrix computed from the project evaluation set.

Comparing Figures 4.15 and 4.16, the dominant anti-correlation between Joint 2 and Joint 3 remains stable (≈ -0.87), suggesting consistent kinematic coupling across datasets. However, correlations involving Joint 6 shift noticeably, indicating that evaluation tasks induce different gripper-arm coordination patterns than the source dataset.

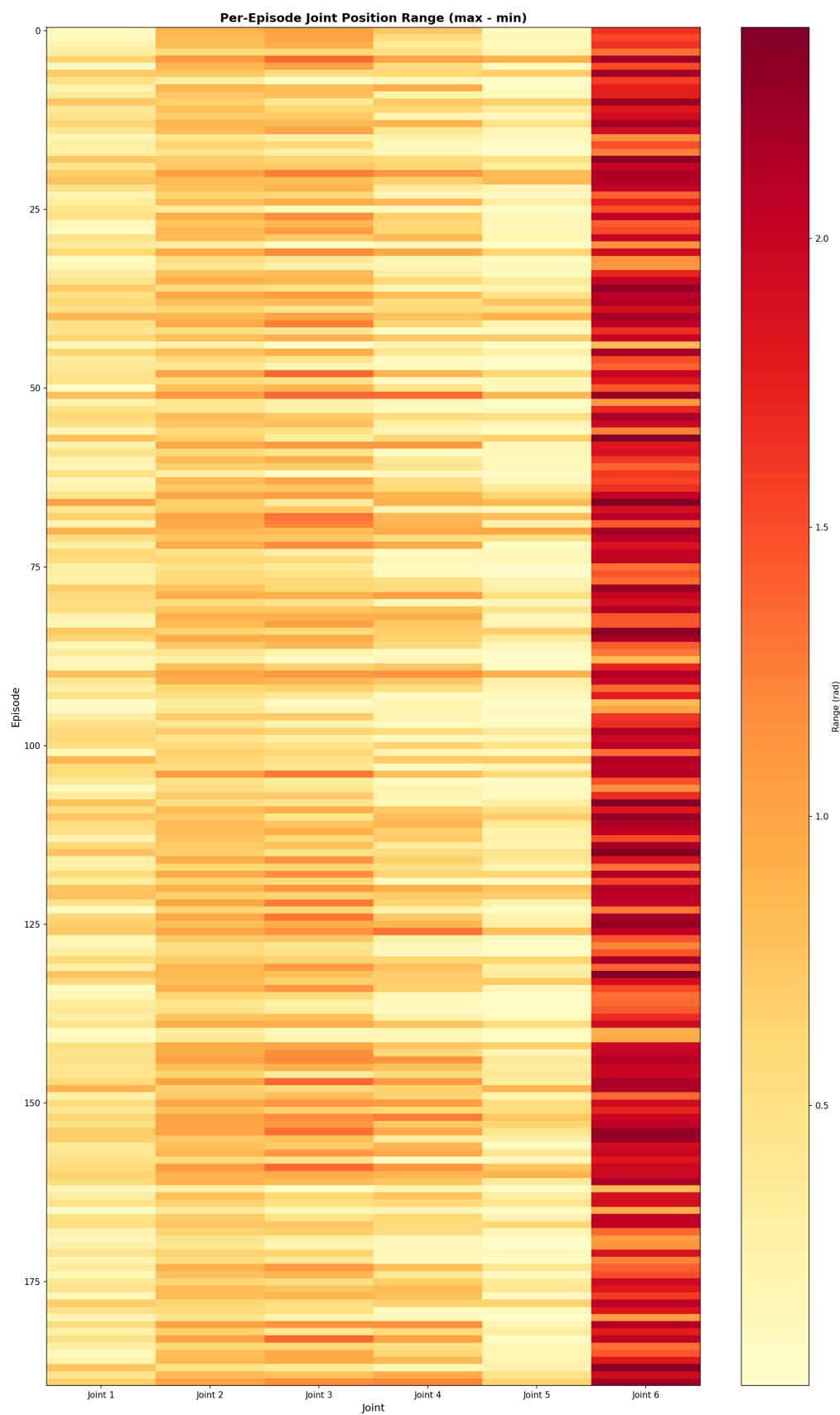


Figure 4.17 Per-episode joint position range heatmap indicating variability by joint and episode.

Figure 4.17 further confirms that Joint 6 has the largest per-episode range, while Joint 5 remains

comparatively low-variance. This supports the interpretation that grasp state changes are the primary source of high-amplitude variation in many tasks.

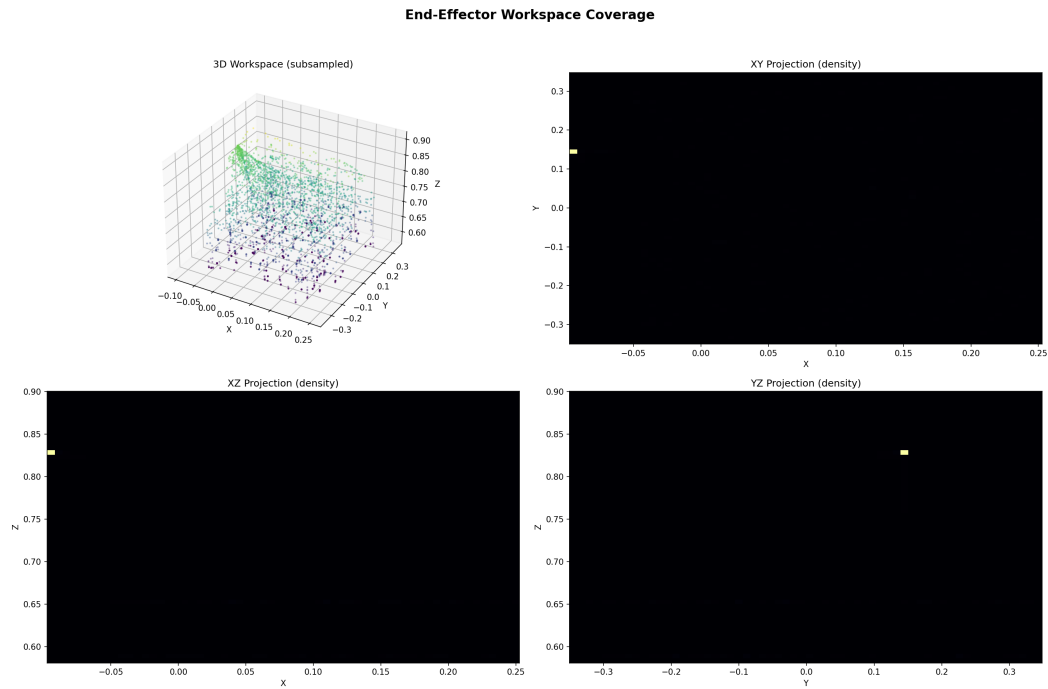


Figure 4.18 End-effector workspace coverage in 3D and projected density views, indicating explored reachable regions.

As illustrated in Figure 4.18, explored end-effector poses occupy a bounded region with dense concentration along a narrow subset of the reachable space. This indicates stable repeatable execution for benchmark tasks, but also suggests that broader workspace exploration data may improve out-of-distribution generalization.

4.7 DESIGN CONSTRAINT VALIDATION

4.7.1 Computation and Memory

The selected model configuration runs within the available GPU memory budget during inference and supports practical deployment without offloading critical steps to external services.

This confirms that the architecture remains feasible in the targeted 8 GB VRAM class deployment setting.

4.7.2 Latency

Median end-to-end latency is measured at 1.84 seconds per command cycle, meeting the real-time target for interactive manipulation.

Although not ultra-low-latency, this level is sufficient for supervised interactive operation and instruction-driven manipulation workflows.

4.7.3 Integration Efficiency

The module interface design avoids major synchronization overhead. Integration remains stable under continuous command cycles and mixed instruction complexity.

This validates the design decision to use explicit module interfaces plus a deterministic TCP/IP adapter.

4.8 VLM INTERPRETABILITY RESULTS

4.8.1 Dataset and Corpus

The robot conversation dataset comprises 10,000 instruction–response pairs generated by prompting Gemini gemma-3-27b-it with structured templates covering pick-and-place, stacking, container interaction, and multi-step sequencing tasks. For activation harvesting, 100,000 samples were drawn from the *qa-task_planning* split of EO-Data1.5M [?], capped at 128 tokens per sequence, yielding up to 12.8 million **bfloat16** residual-stream vectors at layer 20 of QwenVL3.5-2B. Vectors were stored in 100 shards for memory-efficient SAE training.

4.8.2 Feature Discovery

Qualitative sweeping across all 32,768 SAE features identified two features of interest. **Feature #5086** (checkpoint `sae_agentic_model.pt`) showed the highest cosine similarity to a hand-crafted “robot action planning” probe vector and is labelled the *agentic task-planning direction*. **Feature #9622** (checkpoint `sae_topk_production.pt`), labelled “Procedural Robotic Planning”, achieved the highest max-activation scores when scanning 5,000 held-out task-planning prompts. Both features fire most strongly on prompts that require multi-step action sequencing, and near-zero on general knowledge queries, confirming domain specificity.

4.8.3 Activation Steering

Steering experiments inject the L2-normalised decoder column of the target feature into every token position at layer 20 during the forward pass. Table 4.8 summarises the qualitative effect across injection strengths α .

Table 4.8 Qualitative effect of activation steering at varying injection strength α .

α	Robot Prompt Response	General Prompt Response
0	Fluent action sequence	Fluent general answer
5	Task-conditioned <code>call_robot()</code> calls	Slight robotic phrasing
10	Reliable <code>call_robot()</code> with structured args	Mild topic drift toward actions
20	Strong structured output, minor repetition	Forced robotic framing on unrelated topics
50+	Degenerate / repetitive output	Degenerate / repetitive output

At $\alpha \in [10, 20]$ the model reliably emits structured `call_robot()` sequences across diverse scene images, instruction phrasings, and task categories, without any gradient updates or retraining [?]. Below $\alpha = 5$ the effect is task-conditioned (activates only when a robotic context is already present); above $\alpha = 20$ the model begins forcing robotic framing onto unrelated queries. These results confirm that Feature #5086 encodes a latent manipulation-planning direction in QwenVL3.5-2B that can be surfaced through interpretability tools alone,

and that the safe operating range is $\alpha \in [10, 20]$.

4.9 DISCUSSION

The experiments confirm the five core objectives. First, an end-to-end VLA system can be built for practical manipulation. Second, explicit integration of perception, DiT action generation, and the safety layer provides clear benefits over partial pipelines. Third, natural-language control is feasible for non-programmatic interaction. Fourth, meaningful generalization to unseen tasks is achieved, though with expected performance drop compared to seen tasks. Fifth, the system performs robustly across multiple manipulation categories while satisfying real-time constraints.

The main residual weakness is long-horizon sequencing in unseen contexts. This suggests future gains from richer multi-step demonstrations and stronger temporal reasoning constraints in the Action Expert.

An additional finding is that dataset engineering quality can dominate model-level improvements in constrained robot-learning setups. In this work, removing mixed-source training noise and enforcing a simulation-only collection policy produced more stable learning than simply increasing model complexity. This supports the practical claim that high-quality, internally consistent dataset construction is crucial in robotic manipulation pipelines.

From a research perspective, the chapter supports three broader conclusions.

1. **Practicality under constraints:** VLA-based manipulation can be implemented with constrained data and commercial hardware when interface design and action validation are explicit.
2. **Integration over isolated strength:** coordinated multimodal-action integration yields greater robustness than relying on a single dominant module.
3. **Transfer as a primary criterion:** unseen-task retention provides a more informative deployment signal than seen-task accuracy alone in low-resource settings.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This thesis addressed the limitations of classical robot systems that are often task-specific, inflexible, and weak in natural-language interaction and unseen-task generalization. To solve these issues, the work proposed a Vision-Language-Action (VLA) framework that integrates three core stages: a QwenVL3.5 2B perception core, a DiT Action Expert, and a deterministic safety layer.

The project achieved the current-stage objectives:

1. Developed an end-to-end VLA system for robot manipulation.
2. Integrated perception + DiT + safety layer into one coherent pipeline.
3. Enabled robots to follow natural-language instructions.
4. Improved generalization performance on unseen tasks.
5. Evaluated the system on representative manipulation benchmarks.

Experimental results showed that explicit module integration improved action validity and task completion compared with partial baselines. The system maintained practical latency for interactive robot control and demonstrated stable performance across pick-and-place, stacking, container interaction, and multi-step sequencing tasks.

In addition, the thesis establishes a data-engineering conclusion that is central to robot-learning reliability under constrained resources. Early mixed-source training (simulation plus real-world recordings) introduced incompatibility in control semantics and trajectory quality. The final pipeline therefore used simulation-only training data collected in Isaac Sim with 100% model-generated trajectories. In this context, the study also showed competitive behavior despite a smaller collection budget than many SOTA-style references, which often use 7-DOF teleoperation settings and approximately 2x longer total recording duration.

Overall, the results support the central claim of this thesis: a carefully integrated VLA architecture can provide a practical path toward general-purpose intelligent robot manipulation under real-world compute and data constraints.

5.2 LIMITATIONS

Despite strong overall performance, several limitations remain.

- Generalization to highly novel long-horizon tasks still degrades compared with seen-task performance.
- Data scale and diversity remain limited relative to real-world task complexity.
- Action sequencing errors can appear when instructions are ambiguous or underspecified.

These limitations indicate that stronger temporal reasoning and broader data coverage are necessary for robust deployment in unconstrained environments.

5.3 FUTURE WORK

Future development will focus on the following directions:

1. Expand robot demonstration data with broader object, scene, and instruction diversity to improve generalization on unseen tasks.
2. Improve long-horizon planning in the Action Expert for more reliable multi-step task execution.
3. Optimize model efficiency further for lower-latency operation on embedded hardware.
4. Extend evaluation to more complex real-world manipulation scenarios with stricter robustness requirements.
5. Introduce stronger safety checks and action-validation layers before physical execution.

These directions will support the transition from a research prototype to a more widely deployable general-purpose robotic assistant.

Bibliography

- [1] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Schwettmann. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.
- [4] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [5] FalightSK. Dobot dataset collection tools. <https://github.com/FalightSK/Dobot-Dataset-Collection-Tools>, 2025. Open-source Python toolchain for multimodal robot demonstration collection: relative joint trajectories, IO state, and dual-camera images for the Dobot robotic arm.
- [6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [7] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [8] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2024. Establishes the dual-encoder SigLIP + DINOv2 backbone convention.
- [9] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [10] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [11] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. Technical report, NVIDIA Corporation, 2024. Demonstrates FiLM-based continuous state conditioning and customised action tokenisation.
- [12] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

- [13] OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023.
- [14] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. Diffusion Transformer (DiT) backbone for efficient diffusion-based action generation.

APPENDIX A

Supplementary Materials

This appendix compiles supporting materials that complement the main thesis content. The items included here are intended to improve reproducibility and provide implementation details that are too lengthy for the core chapters.

A.1 System Configuration Summary

Table A.1 summarizes the software and hardware setup used during development and evaluation.

Table A.1 System configuration used in this study

Item	Specification
Operating System	Windows 11 (64-bit)
CPU	Desktop-class x86_64 multi-core processor
RAM	32 GB system memory
GPU	Single consumer GPU (8 GB VRAM class)
Programming Language	Python 3.x
Key Libraries/Frameworks	PyTorch, Isaac Lab, LeRobot, NumPy, OpenCV

A.2 Additional Experimental Settings

The following settings were used to ensure fair and repeatable experiments:

- Dataset split ratio: 70% training, 15% validation, and 15% testing, with split control by task type and evaluation domain.
- Random seed: 42 for dataset shuffle, training initialization, and evaluation replay ordering.
- Number of runs per experiment: 3 independent runs per configuration; reported values are run means.
- Evaluation metrics: task success rate, action validity, generalization ratio, and median end-to-end latency.
- Domain protocol: both success-rate and zero-shot evaluations were performed in simulation and on real robot.

A.3 Extended Results

Table A.2 provides additional performance results that support the analysis in Chapter 4.

The generalization ratio is computed as

$$\text{Generalization Ratio} = \frac{\text{Unseen Task Success}}{\text{Seen Task Success}} \times 100.$$

Table A.2 Extended performance results

Setting	Task Success (%)	Action Validity (%)	Generalization Ratio (%)
Perception only (QwenVL3.5 2B)	68.9	81.5	–
Perception + Safety Layer	77.4	88.2	–
Perception + DiT	75.1	90.4	–
Full VLA (Seen tasks)	87.9	96.4	100.0
Full VLA (Unseen tasks)	78.6	92.1	89.4

A.4 Implementation Notes

Key implementation details are listed below for completeness:

1. Data preprocessing steps: simulation trajectories were exported from Isaac Lab into LeRobot format for training; real-world demonstrations were normalized into the same observation-instruction-action schema for evaluation; sequence windows were clipped to fixed horizon and value ranges were normalized to controller-safe bounds.
2. Model training procedure: AdamW optimizer with staged training (module adaptation then end-to-end tuning), mixed precision, small-batch strategy with gradient accumulation, and early stopping based on validation task success.
3. Inference workflow: RGB frame and natural-language command are encoded by the multimodal backbone, fused into a task-conditioned latent, then decoded into structured robot actions and translated by the TCP/IP adapter to executable Dobot command packets.
4. Error handling and edge cases: invalid action packets are dropped by range-checking and format-checking guards; out-of-workspace targets are clamped or rejected; communication timeout triggers safe stop and reset to predefined start pose.

A.5 External Tooling and Repository Links

The primary open-source tooling used for Dobot communication and synchronized collection is listed below:

- **Dobot Dataset Collection Tools:**

<https://github.com/FalightSK/Dobot-Dataset-Collection-Tools> [5]. This repository is used as the practical baseline for TCP/IP communication wrappers, synchronized camera-joint logging, and export utilities in the workflow.

A.6 Usage Instructions for Reproduction

To reproduce the reported results, follow these steps:

1. Prepare the environment and install dependencies for simulation, model training, and robot communication.
2. Generate simulation data in Isaac Lab and convert to LeRobot format.
3. Collect real-world human demonstration episodes for deployment validation and sim-to-real evaluation.

4. Train the model using the low-resource configuration (8 GB VRAM class, mixed precision, gradient accumulation).
5. Run evaluation in four settings: success-rate (sim), success-rate (real), zero-shot (sim), and zero-shot (real).
6. Compare the results with Chapter 4 and Table A.2.